

Fintech Loan Analysis

Complete Project Guide

From six CSV files to a lending recommendation: the data, the work in BigQuery and Hex, every piece of code and every chart explained in plain language, a presentation script and the questions the reviewers may ask.

Project	Fintech loan analysis (loan outcomes, default and late-payment patterns, customer segments, lending rules)
Tools	Google BigQuery (SQL), Hex notebook (SQL and Python), pandas, scikit-learn, Plotly, GitHub
Data	270,299 loans and their customers, issued 2012 to 2019 (six CSV files)
Repository	https://github.com/lukatcheishvili/fintech-loan-analysis
Prepared	21 September 2026

How to use this guide. Read Sections 1 and 2 first (project overview and glossary). Sections 3 to 5 explain the data, BigQuery and Hex. Sections 6 to 10 walk through the five tasks: for every piece of code and every chart you will find what it calculates, how it works, what the result shows and the idea to deliver. Section 12 is the presentation script and Section 13 the questions and answers.

Contents

1. About this guide and the project	5
1.1 What this guide is for	5
1.2 The assignment in plain language	5
1.3 What was delivered	6
1.4 How the work flowed, from file to finding	6
1.5 The tools, explained	7
2. Glossary: terms explained in plain language	8
2.1 Loan and banking terms	9
2.2 Data and statistics terms	11
2.3 Technical terms	12
3. The data: a walk through the six CSV files	13
3.1 Where the data comes from	13
3.2 The big picture	13
3.3 customer.csv, column by column	14
3.4 loan.csv, column by column	15
3.5 The four small files	15
3.6 The seven loan statuses, and how we grouped them	16
3.7 What is missing compared with the brief	16
3.8 Data quality problems found, and how each was handled	17
3.9 The snapshot puzzle: why 2016 looks strange	17
4. What was done in BigQuery	18
4.1 Why BigQuery, and what it is	18
4.2 The setup	18
4.3 Loading the six files as tables	18
4.4 Validating the load: did BigQuery receive exactly what was in the files?	19
4.5 The cleaned view: one place for every definition	20
4.6 Validating the view	22
5. What was done in Hex	24
5.1 What Hex is	24
5.2 Connecting Hex to BigQuery	24
5.3 How the notebook is organised	24
5.4 How cells pass data to each other	25
5.5 Problems met in Hex, and how they were solved	25
6. Task 1: Loan outcomes by year	26

6.1 The approach	26
6.2 Query 1: how many loans per year and outcome	26
6.3 Chart 1: loans by outcome and origination year	27
6.4 Query 2: the same data as percentages	27
6.5 Chart 2: the mix of outcomes in each year	28
7. Task 2: Loan performance by year	30
7.1 The approach and the key decision	30
7.2 Query 1: the yearly summary table	30
7.3 Query 2: the same rates in "long" format, for the chart	31
7.4 Chart 3: default rate by origination year, two definitions	32
8. Task 3: Payment behaviour	34
8.1 What could not be done, and what we did instead	34
8.2 Query 1: what "late" looks like among active loans	34
8.3 Query 2: the customers with late payments	35
8.4 Query 3: late rate by interest-rate band and loan term	35
8.5 Chart 4: late rate against interest rate, as bubbles	37
9. Task 4: Customer segmentation	40
9.1 What could not be done, and what we did instead	40
9.2 Step 1: the customer table	40
9.3 Step 2: preparing the six features	41
9.4 Step 3: choosing the number of groups (k)	42
9.5 Step 4: fitting the five groups and describing them	44
10. Task 5: Insights and recommendations	48
10.1 The approach: three ways of looking, so the answer does not rest on one method	48
10.2 Lens 1: default and late rates by characteristic	48
10.3 Lens 2: a predictive model that weighs everything together	51
10.4 Lens 3: how would simple lending rules have performed?	54
10.5 The written answers to the two questions in the brief	55
11. Limitations, adaptations and quality control	57
11.1 The adaptations, in one table	57
11.2 How correctness was protected	57
11.3 What would be done next with more time or data	57
12. Presentation script	58
Part 1. Opening: what I was asked and what I found (about 1 minute)	58
Part 2. The data and the first surprise (about 1.5 minutes)	58
Part 3. What I did in BigQuery (about 1.5 minutes)	58

Part 4. Task 1: outcomes by year (about 1.5 minutes)	58
Part 5. Task 2: default rate, two ways (about 1.5 minutes)	58
Part 6. Task 3: late payments and interest rate (about 1.5 minutes)	59
Part 7. Task 4: customer segments (about 1.5 minutes)	59
Part 8. Task 5: what predicts default, and the rules (about 2 minutes)	59
Part 9. Challenges and limitations (about 1.5 minutes)	59
Part 10. Closing (about 1 minute)	60
13. Questions the reviewers may ask, with answers	61
13.1 The data and the approach	61
13.2 Finance definitions	62
13.3 Charts and findings	63
13.4 Statistics and modelling	63
13.5 Limitations and judgement	65
13.6 Tools, reproducibility and security	66
Appendix A. Helper scripts used before and during loading	67
A.1 Profiling every CSV file	67
A.2 Generating the BigQuery table schemas	68
Appendix B. The GitHub repository	69
Appendix C. Key numbers at a glance	70
C.1 The dataset	70
C.2 Results by task	70

1. About this guide and the project

1.1 What this guide is for

This guide explains, from the beginning, everything that was done in this project: what data we received, how it was loaded and cleaned in Google BigQuery, how the analysis was built in Hex, what every piece of code calculates, what every chart shows, and what the findings mean. It assumes **no background in banking or data science**. Whenever a technical or financial term appears, it is explained in plain language, and every abbreviation is spelled out in brackets the first time it is used. A full glossary follows in Section 2.

The guide has three purposes: (1) to let you **understand** every step, (2) to give you the **complete code and charts** with an explanation next to each one, and (3) to prepare you for the **review call**, with a ready-to-use presentation script (Section 12) and a long list of questions the reviewers may ask, with detailed answers (Section 13).

How every code block and chart is explained

What this calculates: the purpose in one or two sentences.

How it works, step by step: what each important line does, including every formula, explained with a small worked example.

What the result shows: the actual numbers that came out.

The idea to deliver: the single message you want the reviewers to take away from that piece of work.

1.2 The assignment in plain language

A company (a bank) asked candidates to analyse a public dataset about loans given to people in the United States, and to show they can **query, transform and visualise financial data**. The steps were: download the data from Kaggle (a website that hosts public datasets), upload it into Google BigQuery (a cloud database), connect Hex (an online notebook tool) to it, and then complete five tasks:

Task	What was asked (in simple words)
1. Loan outcomes	Count the loans by outcome (still running, fully repaid, or failed) for each year they were created; show as a bar chart.
2. Loan performance	For each year: the average interest rate, the average amount borrowed, and the share of loans that failed (the default rate); show the default-rate trend as a line chart.
3. Payment behaviour	Find customers who paid late, and show how the frequency of late payment relates to the interest rate (scatter plot or heatmap).
4. Customer segmentation	Use Python to split customers into groups (by loan size, delay, or credit usage) and show the groups as a cluster or bubble chart.
5. Insights and recommendation	Write up what predicts a loan going bad, and what lending rules you would flag or change.

After finishing, a call is arranged with three data-literate business users, to whom you explain how you did it, what you found, and which challenges you faced.

1.3 What was delivered

Deliverable	Where it lives	Section of this guide
Six CSV files loaded into BigQuery, validated against the originals	BigQuery dataset fintech_test	4
One cleaned view (v_loans) holding every definition	BigQuery, file sql/00_staging.sql	4.5
Hex notebook with SQL cells, Python cells, charts and written notes for all five tasks	Hex project (private link) and the exported file	5 to 10
Documented assumptions and adaptations	docs/assumptions.md and the top note of the notebook	11
Public GitHub repository with all code and an AGENT.md context file	github.com/lukatcheishvili/fintech-loan-analysis	Appendix B
Presentation script and questions with answers	This guide	12 and 13

1.4 How the work flowed, from file to finding

Stage	What happened	Tool
1. Get the data	Six CSV files downloaded from Kaggle into the local project folder data/raw (kept out of GitHub).	Kaggle, Windows folder
2. Inspect the data	A script profiled every file: row counts, columns, missing values, value ranges. Cross-file checks confirmed how the files relate.	Python (pandas)
3. Load into the database	Each file uploaded as a BigQuery table with an explicit, hand-checked schema (the list of column names and types).	BigQuery console
4. Validate the load	Queries compared row counts, totals and missing-value counts with numbers computed independently from the files.	SQL (Structured Query Language)
5. Clean and define	One view, v_loans, applies every cleaning rule and defines the key concepts (default, late, resolved, ...) once.	SQL
6. Analyse	Five tasks built as cells in a Hex notebook: SQL cells read v_loans, Python cells do statistics and charts.	Hex, SQL, Python
7. Verify	Every result was checked against an independent local calculation before being written up.	Python (pandas)
8. Publish	Code, documentation and context file pushed to a public GitHub repository.	Git, GitHub

1.5 The tools, explained

Tool	What it is, in plain language	How we used it
CSV file (Comma-Separated Values)	A plain text file that stores a table: each line is a row and the values are separated by commas.	The dataset arrived as six CSV files.
Google Cloud project	An account container in Google Cloud that holds resources and billing. Ours is named pave-bank-fintech.	Holds the BigQuery database.
BigQuery	Google's cloud database. You store tables in it and ask questions with SQL; it answers even on very large tables.	Stores the six raw tables and the cleaned view.
SQL (Structured Query Language)	The standard language for asking questions of a database: choose columns, filter rows, group and count.	All the data extraction and cleaning.
Hex	An online notebook: a page made of cells. Some cells run SQL, some run Python, some draw charts, some hold written text.	The place where the analysis and story live.
Python	A general programming language, very common in data analysis.	Statistics, clustering, charts.
pandas	A Python library for working with tables (called DataFrames).	Reshaping and summarising data.
scikit-learn	A Python library of statistical and machine-learning methods.	k-means clustering and logistic regression.
Plotly	A Python library that draws interactive charts.	The bubble charts and comparison charts.
Git and GitHub	Git records the history of a folder of files; GitHub is a website that hosts such folders (repositories) online.	Public copy of the whole project.

2. Glossary: terms explained in plain language

Terms are grouped in three tables. Where a term has a formula, a small worked example is given. "Percentage points" means the simple difference between two percentages: going from 10% to 15% is a rise of 5 percentage points (but a 50% relative increase).

2.1 Loan and banking terms

Term	Meaning	Example from this project
Loan	Money lent to a borrower that must be repaid, normally with interest.	270,299 loans in the data.
Principal	The original amount borrowed, before any interest. In the data: loan_amount.	A \$10,000 loan has a principal of \$10,000. Average principal is \$15,413.
Interest rate	The yearly price of borrowing, as a percentage of the amount owed. The data stores it as a fraction (0.13 means 13%).	Borrowing \$10,000 at 13% costs roughly \$1,300 in interest over the first year on the full balance.
Term	How long the borrower has to repay: 36 months (3 years) or 60 months (5 years) here.	60-month loans are cheaper per month but riskier for the lender.
Instalment (installment)	The fixed monthly payment that covers part of the principal and the interest.	Ranges from \$29.52 to \$1,719.83 in the data (not used in the analysis).
Origination / issue date	The moment the loan was created and the money paid out. Origination year is the year of that moment.	Loans in the data were originated between 2012 and 2019.
Cohort (vintage)	A group of loans created in the same period, so they can be compared at the same age.	"The 2016 cohort" means all 43,368 loans originated in 2016.
Annual income	The borrower's yearly income as stated on the application.	Median is \$66,000; the range is \$34 to \$9,550,000.
Joint application	A loan requested by two people together, so their incomes are combined.	18,794 loans (about 7%).
Home ownership	Whether the borrower pays a mortgage (loan on a home), rents, or owns the home outright.	MORTGAGE 133,354; RENT 105,663; OWN 30,927.
Income verification	Whether the lender checked the stated income against documents.	Verified, Source Verified, Not Verified.
Credit grade (A to G)	The lender's own risk rating given when the loan is created: A is safest, G is riskiest.	Grade A: 57,871 loans; grade G: 1,206 loans.
Underwriting	The lender's process of judging how risky a borrower is and deciding whether to lend and on what terms.	The grade and interest rate are the result of underwriting.
Risk-based pricing	Charging riskier borrowers higher interest rates to compensate for the higher chance of loss.	Median rate by grade climbs from 7.35% (grade A) through 10.99% (B), 14.30% (C), 18.25% (D), 21.70% (E), 24.99% (F) to 28.18% (G).
Current	A loan on which the borrower is paying as agreed.	170,461 loans.
Grace period	A short window after a due date when a late payment is not yet counted as seriously late.	1,638 loans are in a grace period.
Late (delinquent)	A loan with overdue payments. The data splits it into 16-30 days late and 31-120 days late.	802 loans and 3,174 loans.
Default	The borrower has failed to repay according to the agreement. Here it is also a status value.	12 loans have the status Default.
Charged off	An accounting step where the lender stops expecting repayment and writes the loan off as a loss. Treated as default in this project.	17,851 loans.
Fully paid (paid off)	The loan has been repaid completely.	76,361 loans.
Active loan (our definition)	A loan whose story is not finished: Current, in a grace period, or late.	176,075 loans.
Resolved loan (our definition)	A loan with a known final outcome: paid off or defaulted.	94,224 loans.
Default rate	Defaulted loans divided by a chosen set of loans, times 100. We show two versions: against all loans, and against resolved loans only.	2016: 1,122 defaulted / 43,368 loans = 2.6%, or 1,122 / 6,037 resolved = 18.6%.
Late rate	Late loans divided by active loans, times 100. Only open loans can be late, so they are the right base.	5,614 late / 176,075 active = 3.19%.
Loan-to-income ratio	Loan amount divided by annual income. It says how big the loan is relative to one year of earnings.	\$10,000 loan and \$50,000 income: 10,000 / 50,000 = 0.20 (20%).
Balance-to-income ratio	Total current balance owed across all the borrower's accounts (cards, mortgage, ...) divided by annual income. Used as a rough measure of leverage.	\$150,000 balance and \$60,000 income: 2.5.

Term	Meaning	Example from this project
Leverage	How much debt a person carries relative to what they earn or own. More leverage means more risk.	Balance-to-income is our leverage proxy.
Credit utilisation	The share of available credit that a borrower is using: balance divided by credit limit. Not available in this data.	A card with a \$10,000 limit and \$3,000 owed has 30% utilisation.
Payment history	The record of each payment (dates and amounts). Not available in this data.	It would let us measure exactly how many days late a borrower was.
Snapshot date	The date on which each loan's status was recorded.	Statuses in this data seem to be recorded on different dates for different years.
Right-censoring (immature cohort)	Recent loans have not had time to default, so they look safer than they will eventually be.	Only 607 of 51,737 loans from 2019 are recorded as defaulted.

2.2 Data and statistics terms

Term	Meaning	Example
Row, column, table	A table is a grid: each row is one record (here, one loan) and each column holds one kind of value (for example the interest rate).	loan.csv has 270,299 rows and 18 columns.
Key / join	A column used to link two tables: rows with the same key value describe the same thing. A join combines them.	customer_id links a loan to its customer.
NULL (missing value)	A cell with no value.	annual_inc_joint is NULL for 93% of customers.
Aggregation	Turning many rows into one number: count, sum, average, minimum, maximum.	COUNT(*) of loans per year.
Mean and median	The mean is the sum divided by the count. The median is the middle value when sorted; it is not pulled by extreme values.	Mean income \$79,618 versus median \$66,000: a few very high incomes pull the mean up.
Percentile	The value below which a given percentage of the data falls. The 1st and 99th percentiles mark the extreme 1% at each end.	99% of borrowers earn \$276,000 or less.
Quintile	One of five equal-sized groups after sorting; Q1 is the lowest fifth and Q5 the highest fifth.	Interest-rate quintile Q5 holds the 20% of loans with the highest rates.
Outlier	A value far from the rest.	An income of \$34, or \$9.55 million.
Skewness	How lopsided a distribution is. Near 0 is balanced; a large number means a long tail of very large values.	Income skewness fell from 47.2 to 0.07 after preparation.
Logarithm (log1p)	A mathematical function that compresses large values so that differences are measured in proportions. log1p(x) means $\log(1 + x)$, which stays valid at zero.	It turns the gap between \$20k and \$40k into about the same distance as \$100k to \$200k.
Standard deviation and standardisation	Standard deviation measures typical spread around the mean. Standardising subtracts the mean and divides by it: $z = (x - \text{mean}) / \text{standard deviation}$.	After it, every feature has mean 0 and spread 1, so none dominates.
Correlation versus causation	Two things moving together does not prove one causes the other.	High-rate loans default more, but the lender chose those rates because it saw higher risk.
Spread (percentage points)	Highest minus lowest bucket rate for a characteristic; a large spread means the characteristic separates risk well.	Grade: 48.1% - 5.8% = 42.3 points.
Lift	A group's rate divided by the overall rate. 2.0 means twice as risky as average.	Grade G default lift is about 2.5.
Odds and odds ratio	Odds are the chance of an event divided by the chance it does not happen. An odds ratio compares two situations; above 1 means higher odds.	Interest rate odds ratio 1.63: one standard deviation higher multiplies the odds of default by 1.63.
Train / test split	Data is split: the model learns on the training part and is graded on a test part it never saw.	70% training, 30% test.
Logistic regression	A statistical model that predicts the probability of a yes/no event from several characteristics at once.	Predicts default from rate, term, income, ...
AUC (Area Under the Curve)	A score from 0 to 1 for how well a model ranks risky cases above safe ones. Pick one defaulted and one non-defaulted loan: AUC is the chance the model gives the defaulted one the higher risk score. 0.5 is a coin flip.	Default model 0.697; late-payment model 0.674.
Recall ("defaults caught")	Of all real defaults, the share a rule flags.	Grade E-G rule catches 21.8% of defaults.
False alarms ("good loans flagged")	Of all loans that were fine, the share a rule would also flag. This is the cost of the rule.	Grade E-G rule flags 7.4% of good loans.
k-means clustering	A method that groups similar rows: it picks k group centres, assigns each row to the nearest centre, moves the centres, and repeats until stable.	k = 5 groups of customers.
Inertia	Total squared distance from each row to its group centre; smaller means tighter groups. It always falls as k grows.	1,212,800 at k = 2 down to 675,593 at k = 8.
Silhouette	For each row, how close it is to its own group compared with the nearest other group. From -1 to 1; higher is better; near 0.2 means overlapping groups.	About 0.20 for every k from 3 to 8.
Random seed	A fixed starting number that makes random steps repeatable.	random_state=42 in the code.
Stability (adjusted Rand index)	How similar two groupings are: 1.0 means identical. Used to check that clustering is repeatable across seeds.	k = 5: 0.993.

2.3 Technical terms

Term	Meaning
Query	A written request to a database, for example "count the loans per year".
View	A saved query that behaves like a table. <code>v_loans</code> is our view: it always shows the cleaned version of the raw tables.
Schema	The list of a table's columns and the type of each (text, whole number, decimal, true/false, date).
Dataset (in BigQuery)	A folder that holds tables. Ours is called <code>fintech_test</code> .
Service account	A robot identity in Google Cloud that programs use to log in. Ours, <code>hex-reader</code> , lets Hex read (but not change) the data.
IAM role (Identity and Access Management)	A named bundle of permissions given to an identity, for example "BigQuery Data Viewer" (may read data).
Notebook and cell	A notebook is a document made of cells; each cell holds SQL, Python, a chart, or text, and shows its result underneath.
Result variable	The name Hex gives to a SQL cell's output so that later cells can use it, for example <code>task4_customers</code> .
DataFrame	The pandas object that holds a table in Python.
Kernel	The running Python session behind the notebook; restarting it clears everything in memory.
Repository, commit, branch	A repository is a project folder tracked by Git. A commit is a saved snapshot with a message. A branch is a line of history (<code>main</code> is the default).
Environment file (<code>.env</code>)	A private text file for secrets such as tokens. It must never be published, so it is listed in <code>.gitignore</code> .
<code>AGENT.md</code>	A context file that tells an AI assistant what the project is and what has been done, so a new session can continue seamlessly.

3. The data: a walk through the six CSV files

3.1 Where the data comes from

The data was downloaded from Kaggle, the dataset called "BigQuery Fintech Dataset". It contains **six CSV (Comma-Separated Values) files**. No data dictionary (a document explaining every column) came with it. The meaning of each column below is therefore **inferred from the column names and from the values themselves**. The column names and value formats closely resemble the well-known public LendingClub loan data (LendingClub was a US online lender), which suggests that is the origin, but that is a hypothesis and is not stated anywhere in the files.

Before loading anything, a small script profiled every file (row counts, column types, missing values, smallest and largest values, most common values) so that decisions were based on what the data really contains and not on assumptions. Its full code is in Appendix A.

3.2 The big picture

File	Rows	Columns	What it is
customer.csv	270,299	11	One row per borrower: job, income, home ownership, location, balances.
loan.csv	270,299	18	One row per loan: amount, rate, term, grade, dates, purpose and current status. The core table.
loan_with_region.csv	270,299	3	A derived helper file: loan_id, loan_amount and the US region of the loan.
loan_count_by_year.csv	8	2	A derived helper file: the number of loans per origination year.
loan_purposes.csv	13	1	A lookup list of the 13 loan purposes.
state_region.csv	52	3	A lookup that maps each US state to a sub-region and a region (contains one stray header row).

How the files connect. loan.csv and customer.csv are linked by the column customer_id. Every one of the 270,299 loans matches exactly one customer and every customer matches exactly one loan (we tested the match in both directions): it is a strict **one-to-one** relationship. That has an important consequence: **each customer has only one loan**, so "total loan amount per customer" is simply that loan's amount, and we can never see how one person behaves across several loans. loan_with_region.csv links to loan.csv through loan_id, and state_region.csv links through the state code.

Link	Joined on	Result of the check
loan to customer	customer_id	270,299 matches in both directions; no unmatched rows; each id unique in both files
loan to loan_with_region	loan_id	270,299 matches; loan amounts identical; regions agree 100% with the state lookup
loan to state_region	state	All 51 states in loan are found in the lookup
loan vs loan_count_by_year	issue_year	The eight yearly counts equal our own count of loan.csv, year by year

3.3 customer.csv, column by column

Column	Type	Meaning (inferred)	What we found
customer_id	text	A hashed (scrambled) identifier for the borrower.	Unique for all 270,299 rows. It is stored as text that looks like b"\xa0\xe6..." (the printed form of raw bytes), so it is unreadable and used only as a join key.
emp_title	text	The borrower's job title.	88,934 different titles; 23,658 blank (8.75%). Not used in the analysis.
emp_length	text	How long the borrower has held the job.	Values such as "10+ years" and "< 1 year"; the text n/a (18,745 rows) is a missing-value code.
home_ownership	text	Mortgage, rent, or own outright.	MORTGAGE 133,354; RENT 105,663; OWN 30,927; ANY 349; OTHER 4; NONE 2.
annual_inc	decimal	Stated yearly income in dollars.	Median \$66,000; mean \$79,618; minimum \$34; maximum \$9,550,000; 1st percentile \$17,000; 99th percentile \$276,000.
annual_inc_joint	decimal	Combined income for joint applications.	Blank for 251,505 customers (93.05%); filled for exactly the 18,794 joint-application loans.
verification_status	text	Whether the income was checked by the lender.	Source Verified 105,373; Not Verified 97,851; Verified 67,075.
zip_code	text	First three digits of the postal code plus "xx".	887 different values, for example 010xx.
addr_state	text	The borrower's US state.	51 values; identical to loan.state for every row.
avg_cur_bal	decimal	Average current balance per account of the borrower.	Median \$7,376; maximum \$623,229. Not used directly.
Tot_cur_bal	decimal	Total current balance across all the borrower's accounts (cards, mortgage, ...).	Median \$80,478; mean \$144,693; maximum \$9,971,659. Used to build the balance-to-income ratio.

3.4 loan.csv, column by column

Column	Type	Meaning (inferred)	What we found
loan_id	whole number	Unique number of the loan.	1 to 270,299 with no gaps. The readable handle we use for each loan and customer.
customer_id	text	Link to customer.csv.	Unique for all rows.
loan_status	text	Where the loan stands now.	Seven values (Section 3.6).
loan_amount	whole number	Principal: dollars borrowed.	\$1,000 to \$40,000; mean \$15,413; median \$13,200; total \$4,166,072,400 (about \$4.17 billion).
state	text	US state of the loan.	51 values; California 37,024, Texas 22,541, New York 21,854, Florida 19,846, Illinois 10,845.
funded_amount	whole number	Dollars actually funded.	Identical to loan_amount on every row.
term	text	Repayment period.	" 36 months" (189,772 loans) and " 60 months" (80,527). Both values carry a leading space.
int_rate	decimal	Yearly interest rate as a fraction.	0.0531 to 0.3099, i.e. 5.31% to 30.99%; mean 13.08%; median 12.61%.
installment	decimal	Monthly payment in dollars.	\$29.52 to \$1,719.83; mean \$453.92. Not used in the analysis.
grade	text	The lender's risk grade at origination, A (safest) to G (riskiest).	A 57,871 (21.4%); B 79,072 (29.3%); C 75,840 (28.1%); D 38,876 (14.4%); E 13,429 (5.0%); F 4,005 (1.5%); G 1,206 (0.4%).
issue_d	text	Month the loan was issued, such as Jun-13.	89 different months, from 2012 to Dec-19.
issue_date	text	Same information again.	Identical to issue_d in the final files (a redundant column).
issue_year	whole number	Year the loan was issued.	2012 to 2019.
pymnt_plan	true/false	Whether a special payment plan was arranged.	TRUE for only 40 loans.
type	text	Application type.	Five spellings of three things: Individual 169,493 + INDIVIDUAL 81,709; Joint App 17,931 + JOINT 863; DIRECT_PAY 303.
purpose	text	What the loan is for.	13 values (Section 3.5).
description	text	Free text written by the borrower.	5,631 different texts; 2,343 blank. Not used.
notes	text	-	The same text "desc" on every row: useless, dropped.

3.5 The four small files

loan_with_region.csv repeats loan_id and loan_amount and adds a region. South has 97,683 loans, West 69,137, Northeast 55,014 and Midwest 48,465. We use it only as a cross-check of the region we derive ourselves. **loan_count_by_year.csv** holds the yearly counts (2,594 in 2012, 13,460 in 2013, 23,453 in 2014, 41,919 in 2015, 43,368 in 2016, 44,435 in 2017, 49,333 in 2018, 51,737 in 2019), which we used to confirm our own counts. **state_region.csv** maps 51 states to 9 sub-regions and 4 regions; it also contains one stray row that repeats the column names (state, subregion, region) inside the data, which must be removed.

loan_purposes.csv lists the 13 purposes:

Purpose	Loans	Share	Purpose	Loans	Share
debt_consolidation	152,285	56.3%	moving	1,744	0.6%
credit_card	64,477	23.9%	renewable_energy	156	0.1%
home_improvement	17,601	6.5%	wedding	91	0.03%
other	16,279	6.0%	house	1,786	0.7%
major_purchase	5,584	2.1%	vacation	1,863	0.7%
medical	3,173	1.2%	small_business	2,615	1.0%
car	2,645	1.0%			

More than 80% of the loans are taken either to combine existing debts into one (debt consolidation) or to pay off credit cards. Purpose was available but was not needed for the five tasks.

3.6 The seven loan statuses, and how we grouped them

The brief spoke of three outcomes: active, paid_off and defaulted. The data has seven statuses. We mapped them into the three groups as follows (the mapping is written once, in the cleaned view, so every task uses the same one):

loan_status	Loans	Group used in the analysis
Current	170,461	active
Fully Paid	76,361	paid_off
Charged Off	17,851	defaulted
Late (31-120 days)	3,174	active
In Grace Period	1,638	active
Late (16-30 days)	802	active
Default	12	defaulted

- **paid_off** = Fully Paid: 76,361 loans (28.3%).
- **defaulted** = Charged Off + Default: 17,851 + 12 = 17,863 loans (6.6%).
- **active** = Current + In Grace Period + Late (16-30 days) + Late (31-120 days): 170,461 + 1,638 + 802 + 3,174 = 176,075 loans (65.1%).
- The three groups add up to 270,299, the full dataset.

We also created two helper ideas used throughout. **Resolved** loans are those with a known outcome (paid off or defaulted): 76,361 + 17,863 = 94,224 loans. **Late** loans are the ones in a grace period or 16-30 or 31-120 days late: 1,638 + 802 + 3,174 = 5,614 loans.

3.7 What is missing compared with the brief

The brief assumed	The data has	Consequence for the tasks
Tables named loans, payments, customers, transactions	customer, loan and four small helper files. No payments table, no transactions table.	Task 3 cannot "join loans and payments". The only join possible is loan to customer.
Statuses active, paid_off, defaulted	Seven statuses.	We mapped them into the three groups (Section 3.6).
A delay or days-late measure per customer	No payment dates, no days-late column.	"Average delay" cannot be computed (Tasks 3 and 4).
Credit utilisation	No credit limits.	Cannot be computed. We used balance divided by income as a rough leverage proxy instead.
Several loans per customer	Exactly one loan per customer.	"Late payment frequency per customer" cannot be a count; it became the share of loans that are late within a group.

3.8 Data quality problems found, and how each was handled

Problem	How we found it	How it was handled
customer_id is a hashed byte string stored as text	Raw file peek: values look like b'\xa0\xe6...'; some contain commas and quotes.	Loaded as plain text, exactly as written; used only to join; loan_id used as the readable handle.
The text n/a means "missing" in emp_length (18,745 rows)	Profile: pandas treated it as missing, BigQuery would keep it as text.	Turned into a true missing value (NULL) in the view.
A stray header row inside state_region.csv	The states column contained the word "state"; 52 rows instead of 51.	Filtered out in the view.
Application type has five spellings	Distinct values: Individual, INDIVIDUAL, Joint App, JOINT, DIRECT_PAY.	Collapsed into three: Individual, Joint, Direct Pay.
Leading space in term (" 36 months")	Value counts showed two spaces-prefixed values.	Number extracted: 36 or 60.
Duplicate and constant columns	issue_date equals issue_d; notes is always "desc"; funded_amount equals loan_amount.	Dropped or ignored.
Extreme values	Income from \$34 to \$9,550,000; loan-to-income up to 175.	Capped at the 1st and 99th percentile for clustering and modelling; nobody deleted.
Joint income mostly empty	93% blank.	Understood as by design (only joint applications have it); only the main applicant's income is used.
Files changed while we were reading them	A file size changed between two listings; one read was briefly denied by Windows.	All files were re-profiled and re-checked after they stopped changing.

3.9 The snapshot puzzle: why 2016 looks strange

When we counted the outcomes for each origination year, 2016 stood out: **86.1% of 2016 loans are still "active"**, and this is the same for 36-month loans (86.1%) and 60-month loans (86.1%). The latest loan in the data was issued in December 2019, so a 36-month loan issued in 2016 should have finished by the end of 2019 at the latest. Yet 86% of them are recorded as still open. 2014 is also unusual: 29% of its 36-month loans are still open, while for 2013 and 2015 the figure is 0%.

Our working explanation (**a hypothesis, not proven**) is that the statuses were not all recorded on the same date: some years were probably recorded at one point in time and some at another. This matters because it makes year-to-year comparisons of default counts unreliable, which is why Task 2 shows the default rate in two ways. It is a point worth stating openly to the reviewers.

Year	Share of loans still active, 36-month	Share still active, 60-month
2012	0.0%	23.5%
2013	0.0%	33.7%
2014	29.0%	48.4%
2015	0.0%	30.3%
2016	86.1%	86.1%
2017	70.9%	76.4%
2018	92.1%	93.7%
2019	90.2%	93.3%

4. What was done in BigQuery

4.1 Why BigQuery, and what it is

BigQuery is Google's cloud database. The brief asked for the data to be placed in it so that the analysis can be done with SQL (Structured Query Language) directly on the stored tables, in the same way a bank's analysts would work on a data warehouse. Working in a database rather than in spreadsheet files brings three benefits: the numbers are reproducible (anyone can run the same query and get the same answer), the definitions can be stored once and reused, and it scales to far larger data than this.

4.2 The setup

Item	Value	Explanation
Google Cloud project	pave-bank-fintech	The account container that owns the database and any costs.
Dataset	fintech_test	A folder inside the project that holds the tables. Created in the US multi-region.
Location	US (multi-region)	The physical location of the data cannot be changed later, so it was chosen once and kept consistent.
Billing	Not required for this size	The data is only about 115 MB in total, well within the free usage limits.

4.3 Loading the six files as tables

Each CSV file was uploaded through the BigQuery console (the web interface) into a table of the same name: customer, loan, loan_with_region, loan_count_by_year, loan_purposes and state_region. Two design decisions are worth explaining.

Decision 1: an explicit schema instead of automatic detection. BigQuery can guess column types, but guesses go wrong (for example a date such as Jun-13 is guessed as text, and an identifier can lose characters). We generated the schema for every table from the real file headers with a small script (Appendix A) and pasted it into the console. The script also checks that the columns are listed in exactly the order of the file, because BigQuery matches columns by *position*, not by name.

Decision 2: load the raw data unchanged, and clean it later in a view. The raw tables are never edited. This is standard good practice: you can always go back to the original, and every cleaning rule is visible in one place (Section 4.5).

Table	Column types (INTEGER = whole number, FLOAT = decimal, STRING = text, BOOLEAN = true/false)
customer	customer_id STRING; emp_title STRING; emp_length STRING; home_ownership STRING; annual_inc FLOAT; annual_inc_joint FLOAT; verification_status STRING; zip_code STRING; addr_state STRING; avg_cur_bal FLOAT; Tot_cur_bal FLOAT
loan	loan_id INTEGER; customer_id STRING; loan_status STRING; loan_amount INTEGER; state STRING; funded_amount INTEGER; term STRING; int_rate FLOAT; installment FLOAT; grade STRING; issue_d STRING; issue_date STRING; issue_year INTEGER; pymnt_plan BOOLEAN; type STRING; purpose STRING; description STRING; notes STRING
loan_with_region	loan_id INTEGER; loan_amount INTEGER; region STRING
loan_count_by_year	issue_year INTEGER; loan_count INTEGER
loan_purposes	purpose STRING
state_region	state STRING; subregion STRING; region STRING

Dates such as Jun-13 cannot be loaded as a date type directly, so they stay as text in the raw table and are converted to a real date in the view. Postal codes are kept as text because they are labels, not quantities (010xx is not a number).

Upload settings used for every table: source = upload a local file; format = CSV; schema = pasted as text (automatic detection off); header rows to skip = 1; number of errors allowed = 0 (so that any bad row makes the

upload fail loudly instead of silently dropping data); quoted newlines allowed. Two small stumbles are recorded as lessons: the schema box needs the JSON *contents* and not the file path, and the console accepts the older type names (INTEGER, FLOAT, BOOLEAN) but not the newer aliases (INT64, FLOAT64, BOOL).

4.4 Validating the load: did BigQuery receive exactly what was in the files?

Uploading without checking is a risk: rows can be dropped, numbers can be altered, blanks can be read differently. So a set of queries was written whose **expected answers were calculated independently in Python from the original files**. If BigQuery returns the same numbers, the load is faithful. The queries are in the file `sql/01_validate_load.sql`:

Validation queries for the raw tables (`sql/01_validate_load.sql`)

```
-- Step 12: validate the raw load. Run in BigQuery Studio after all 6 tables are uploaded.
-- "expected" values come from the local CSVs (scripts/profile_csvs.py + pandas checks).
-- Every `diff` column should be 0 (or ~0 for float sums: BigQuery may add in a different order).

-- A. Row counts -----
SELECT 'customer' AS tbl, COUNT(*) AS actual, 270299 AS expected, COUNT(*) - 270299 AS diff FROM
  » `pave-bank-fintech.fintech_test.customer`
UNION ALL SELECT 'loan', COUNT(*), 270299, COUNT(*) - 270299 FROM `pave-bank-fintech.fintech_test.loan`
UNION ALL SELECT 'loan_with_region', COUNT(*), 270299, COUNT(*) - 270299 FROM `pave-bank-fintech.fintech_test.loan_with_region`
UNION ALL SELECT 'loan_count_by_year', COUNT(*), 8, COUNT(*) - 8 FROM `pave-bank-fintech.fintech_test.loan_count_by_year`
UNION ALL SELECT 'loan_purposes', COUNT(*), 13, COUNT(*) - 13 FROM `pave-bank-fintech.fintech_test.loan_purposes`
UNION ALL SELECT 'state_region', COUNT(*), 52, COUNT(*) - 52 FROM `pave-bank-fintech.fintech_test.state_region`; -- 52 = 51
  » states + 1 stray header row (filtered later)

-- B. Content checks on loan -----
SELECT check_name, actual, expected, actual - expected AS diff
FROM (
  SELECT 'SUM(loan_amount)' AS check_name, CAST(SUM(loan_amount) AS FLOAT64) AS actual, 4166072400 AS expected FROM
    » `pave-bank-fintech.fintech_test.loan`
  UNION ALL SELECT 'SUM(funded_amount)', CAST(SUM(funded_amount) AS FLOAT64), 4166072400 FROM
    » `pave-bank-fintech.fintech_test.loan`
  UNION ALL SELECT 'AVG(int_rate) (6dp)', ROUND(AVG(int_rate), 6), 0.130765 FROM
    » `pave-bank-fintech.fintech_test.loan`
  UNION ALL SELECT 'SUM(installment) (2dp)', ROUND(SUM(installment), 2), 122693899.23 FROM
    » `pave-bank-fintech.fintech_test.loan`
  UNION ALL SELECT 'NULL description', COUNTIF(description IS NULL), 2343 FROM
    » `pave-bank-fintech.fintech_test.loan`
  UNION ALL SELECT 'pymnt_plan = TRUE', COUNTIF(pymnt_plan), 40 FROM
    » `pave-bank-fintech.fintech_test.loan`
  UNION ALL SELECT 'DISTINCT customer_id', COUNT(DISTINCT customer_id), 270299 FROM
    » `pave-bank-fintech.fintech_test.loan`
);

-- C. Content checks on customer -----
SELECT check_name, actual, expected, actual - expected AS diff
FROM (
  SELECT 'SUM(annual_inc) (2dp)' AS check_name, ROUND(SUM(annual_inc), 2) AS actual, 21520664782.41 AS expected FROM
    » `pave-bank-fintech.fintech_test.customer`
  UNION ALL SELECT 'SUM(Tot_cur_bal) (2dp)', ROUND(SUM(Tot_cur_bal), 2), 39110405893.0 FROM
    » `pave-bank-fintech.fintech_test.customer`
  UNION ALL SELECT 'NULL emp_title', COUNTIF(emp_title IS NULL), 23658 FROM
    » `pave-bank-fintech.fintech_test.customer`
  UNION ALL SELECT 'NULL annual_inc_joint', COUNTIF(annual_inc_joint IS NULL), 251505 FROM
    » `pave-bank-fintech.fintech_test.customer`
  UNION ALL SELECT "emp_length = 'n/a'", COUNTIF(emp_length = 'n/a'), 18745 FROM
    » `pave-bank-fintech.fintech_test.customer`
  UNION ALL SELECT 'DISTINCT customer_id', COUNT(DISTINCT customer_id), 270299 FROM
    » `pave-bank-fintech.fintech_test.customer`
);

-- D. Join integrity: every loan has exactly one customer, and vice versa (expect 0 and 0)
SELECT
  (SELECT COUNT(*) FROM `pave-bank-fintech.fintech_test.loan` l
   LEFT JOIN `pave-bank-fintech.fintech_test.customer` c ON l.customer_id = c.customer_id WHERE c.customer_id IS NULL) AS
    » loans_without_customer,
  (SELECT COUNT(*) FROM `pave-bank-fintech.fintech_test.customer` c
   LEFT JOIN `pave-bank-fintech.fintech_test.loan` l ON c.customer_id = l.customer_id WHERE l.customer_id IS NULL) AS
    » customers_without_loan;

-- E. True status vocabulary (expect the 7 values profiled locally)
SELECT loan_status, COUNT(*) AS loans
FROM `pave-bank-fintech.fintech_test.loan`
GROUP BY loan_status
ORDER BY loans DESC;
```

What this calculates

It compares what is stored in BigQuery with what the source files contain: the number of rows in every table, the sums of key money columns, the counts of blanks, the number of unique customer identifiers, and whether every loan finds its customer and vice versa.

How it works, step by step

- **Section A (row counts)** counts the rows of each table and subtracts the expected count; the difference (diff) must be 0. The state_region table is expected to have 52 rows: 51 states plus the one stray header row.
- **Section B (loan checks)** sums loan_amount and funded_amount (expected 4,166,072,400 for each), averages the interest rate (expected 0.130765), sums the instalments (122,693,899.23), counts blank descriptions (2,343), loans with a payment plan (40) and distinct customers (270,299).
- **Section C (customer checks)** sums income (21,520,664,782.41) and balances (39,110,405,893), counts blank job titles (23,658), blank joint income (251,505), the text n/a in emp_length (18,745) and distinct customers (270,299).
- **Section D (join integrity)** uses a LEFT JOIN: it keeps every row on the left and shows NULL where there is no match on the right. Counting the rows where the right side is NULL gives the loans without a customer and the customers without a loan; both must be 0.
- **Section E** lists the distinct loan statuses with their counts, to confirm the seven expected values.
- The reported difference is *actual minus expected*. For decimal sums a tiny difference can appear because computers add decimals in a different order; we accept a difference that small.

What the result shows

Every check matched: all six row counts, every sum, every blank count, and both join checks returned 0 unmatched rows. The count of blank descriptions being exactly 2,343 confirmed that empty cells were loaded as missing values.

The idea to deliver

The data in BigQuery is exactly the data in the files. Any number in the analysis can be traced back to the source with confidence.

4.5 The cleaned view: one place for every definition

A **view** is a saved query that looks like a table. We created one, called **v_loans**, that joins the three tables needed (loan, customer, state_region), cleans the messy fields and defines the key concepts. **Every task then reads from v_loans**, so the definition of "default" or "late" exists in exactly one place and cannot drift between tasks. The full query is in the file sql/00_staging.sql:

The cleaned view v_loans (sql/00_staging.sql)

```
-- Step 13: staging view. Single source of truth for cleaning rules and the
-- status_group / is_default / is_late definitions (see docs/assumptions.md).
-- Raw tables are never modified. Loans are LEFT JOINed so no loan can silently disappear.

CREATE OR REPLACE VIEW `pave-bank-fintech.fintech_test.v_loans` AS
SELECT
  l.loan_id,
  l.customer_id,

  -- Outcome definitions -----
  l.loan_status,
  CASE
    WHEN l.loan_status = 'Fully Paid' THEN 'paid_off'
    WHEN l.loan_status IN ('Charged Off', 'Default') THEN 'defaulted'
    ELSE 'active' -- Current, In Grace Period, Late (16-30 days), Late (31-120 days)
  END AS status_group,
  l.loan_status IN ('Charged Off', 'Default') AS is_default,
  -- Task 3 workaround: no payments table, so "late" is a snapshot of current status
  l.loan_status IN ('In Grace Period', 'Late (16-30 days)', 'Late (31-120 days)') AS is_late,
  -- Outcome known (paid or defaulted); use for default rates that ignore still-open loans
  l.loan_status IN ('Fully Paid', 'Charged Off', 'Default') AS is_resolved,

  -- Loan terms -----
  l.loan_amount,
```

```

1.funded_amount,
1.int_rate, -- fraction: 0.1299 = 12.99%
1.installment,
CAST(REGEXP_EXTRACT(1.term, r'(\d+)') AS INT64) AS term_months, -- ' 36 months' -> 36
1.grade,
PARSE_DATE('%b-%y', 1.issue_d) AS issue_date, -- 'Jun-13' -> 2013-06-01
1.issue_year,
CASE UPPER(TRIM(1.type))
  WHEN 'INDIVIDUAL' THEN 'Individual'
  WHEN 'JOINT' THEN 'Joint'
  WHEN 'JOINT APP' THEN 'Joint'
  WHEN 'DIRECT_PAY' THEN 'Direct Pay'
  ELSE 1.type
END AS application_type,
1.purpose,
1.pymnt_plan,

-- Geography (state_region has one stray header row, filtered below) -----
1.state,
sr.region,
sr.subregion,

-- Borrower -----
c.emp_title,
NULLIF(c.emp_length, 'n/a') AS emp_length, -- 'n/a' is a missing value
c.home_ownership,
c.verification_status,
c.zip_code,
c.annual_inc,
c.annual_inc_joint, -- NULL unless a joint application
SAFE_DIVIDE(1.loan_amount, c.annual_inc) AS loan_to_income,
c.avg_cur_bal,
c.Tot_cur_bal AS tot_cur_bal
FROM `pave-bank-fintech.fintech_test.loan` AS l
LEFT JOIN `pave-bank-fintech.fintech_test.customer` AS c
  ON l.customer_id = c.customer_id
LEFT JOIN (
  SELECT * FROM `pave-bank-fintech.fintech_test.state_region` WHERE state != 'state'
) AS sr
  ON l.state = sr.state;

```

What this calculates

It builds one row per loan with clean, correctly typed columns, plus new columns that encode the business definitions: the outcome group, whether a loan is defaulted, late or resolved, the term in months, a real issue date, the application type, the region, and the loan-to-income ratio.

How it works, step by step

- **status_group** uses CASE, SQL's if-then-else. Fully Paid becomes paid_off; Charged Off and Default become defaulted; everything else becomes active.
- **is_default, is_late, is_resolved** use IN (...), which is true when the status is in the listed set. is_default = Charged Off or Default. is_late = In Grace Period, Late (16-30 days) or Late (31-120 days). is_resolved = Fully Paid, Charged Off or Default.
- **int_rate** stays a fraction (0.1299 = 12.99%); it is multiplied by 100 whenever a percentage is displayed.
- **term_months**: REGEXP_EXTRACT(term, r'(\d+)') pulls out the first run of digits from " 36 months", giving 36; CAST(... AS INT64) turns the text into a number. This also removes the troublesome leading space.
- **issue_date**: PARSE_DATE('%b-%y', 'Jun-13') reads the text as a date. %b means the three-letter month, %y a two-digit year (00 to 68 are read as 2000 to 2068), so Jun-13 becomes 2013-06-01. The day is not in the text, so BigQuery uses the 1st.
- **application_type**: UPPER(TRIM(type)) removes stray spaces and capitals, then CASE maps INDIVIDUAL to Individual, JOINT and JOINT APP to Joint, and DIRECT_PAY to Direct Pay.
- **emp_length**: NULLIF(emp_length, 'n/a') returns NULL when the value is n/a, so a missing-value code becomes a true missing value.
- **loan_to_income** = SAFE_DIVIDE(loan_amount, annual_inc), the loan amount divided by annual income. Example: a \$10,000 loan and a \$50,000 income give 10,000 / 50,000 = 0.20. SAFE_DIVIDE returns NULL instead of an error if income were 0.
- **The joins**: LEFT JOIN keeps every loan even if a match were missing (so a loan can never silently disappear). The state_region lookup is first filtered with WHERE state != 'state' to remove the stray header row.
- **tot_cur_bal** is Tot_cur_bal renamed with a lower-case first letter for consistency.

What the result shows

The view returns 270,299 rows, one per loan, with 31 clean columns. Raw tables are untouched.

The idea to deliver

All cleaning and every business definition live in one auditable place; every number in the notebook can be traced to these rules.

4.6 Validating the view

The view was checked the same way as the raw tables: 17 counts computed from the source files were compared with the view's results, plus a cross-check of the region against the separate loan_with_region file. The queries are in sql/02_validate_staging.sql:

Validation queries for the view (sql/02_validate_staging.sql)

```
-- Step 13b: validate v_loans. Expected values come from the local CSV analysis
-- (docs/assumptions.md, docs/schema_profile.md). Every diff must be 0.

-- A. Counts and derived columns
SELECT check_name, actual, expected, actual - expected AS diff
FROM (
  SELECT 'rows' AS check_name, COUNT(*) AS actual, 270299 AS expected FROM `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'status_group = paid_off', COUNTIF(status_group = 'paid_off'), 76361 FROM `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'status_group = defaulted', COUNTIF(status_group = 'defaulted'), 17863 FROM `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'status_group = active', COUNTIF(status_group = 'active'), 176075 FROM `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'is_default', COUNTIF(is_default), 17863 FROM `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'is_late', COUNTIF(is_late), 5614 FROM `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'is_resolved', COUNTIF(is_resolved), 94224 FROM `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'NULL issue_date', COUNTIF(issue_date IS NULL), 0 FROM `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'issue_year <> YEAR(issue_date)', COUNTIF(issue_year != EXTRACT(YEAR FROM issue_date)), 0 FROM
    » `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'term_months = 36', COUNTIF(term_months = 36), 189772 FROM `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'term_months = 60', COUNTIF(term_months = 60), 80527 FROM `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'application_type = Individual', COUNTIF(application_type = 'Individual'), 251202 FROM
    » `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'application_type = Joint', COUNTIF(application_type = 'Joint'), 18794 FROM
    » `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'application_type = Direct Pay', COUNTIF(application_type = 'Direct Pay'), 303 FROM
    » `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'NULL region', COUNTIF(region IS NULL), 0 FROM `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'NULL annual_inc (customer join)', COUNTIF(annual_inc IS NULL), 0 FROM `pave-bank-fintech.fintech_test.v_loans`
  UNION ALL SELECT 'NULL emp_length (was n/a)', COUNTIF(emp_length IS NULL), 18745 FROM `pave-bank-fintech.fintech_test.v_loans`
);

-- B. Region derived from state_region must agree with the provided loan_with_region file
-- (expect region_mismatches = 0 and joined_rows = 270299)
SELECT COUNTIF(v.region != r.region) AS region_mismatches, COUNT(*) AS joined_rows
FROM `pave-bank-fintech.fintech_test.v_loans` AS v
JOIN `pave-bank-fintech.fintech_test.loan_with_region` AS r
  ON v.loan_id = r.loan_id;

-- C. Eyeball a few rows
SELECT loan_id, loan_status, status_group, is_default, is_late, loan_amount, int_rate,
  term_months, grade, issue_date, issue_year, application_type, region, emp_length,
  annual_inc, loan_to_income
FROM `pave-bank-fintech.fintech_test.v_loans`
LIMIT 10;
```

What this calculates

It confirms that the cleaning and definitions in the view produce exactly the counts we independently calculated: 270,299 rows; 76,361 paid_off; 17,863 defaulted; 176,075 active; 5,614 late; 94,224 resolved; every date parsed and consistent with issue_year; 189,772 loans of 36 months and 80,527 of 60 months; 251,202 Individual, 18,794 Joint and 303 Direct Pay applications; no missing region; 18,745 missing employment lengths.

How it works, step by step

- Section A stacks 17 checks with UNION ALL; each shows actual, expected and diff.
- The check `issue_year <> YEAR(issue_date)` proves the date parsing is right: the year read from the text date equals the separate issue_year column for every loan.
- Section B joins the view to loan_with_region and counts mismatching regions (expected 0 out of 270,299).
- Section C shows ten sample rows to eyeball.

What the result shows

All 17 differences were 0, the region cross-check had 0 mismatches over 270,299 rows, and a final one-row check confirmed the term counts (189,772 and 80,527) and 0 missing regions.

The idea to deliver

The definitions were tested, not just written: the view is correct before any analysis is built on it.

5. What was done in Hex

5.1 What Hex is

Hex is an online notebook. A notebook is a single page made of **cells**, run from top to bottom, where each cell shows its result directly underneath. Hex mixes four kinds of cell, and this project uses all of them:

Cell type	What it does	Used for
SQL cell	Sends a SQL query to BigQuery and stores the answer as a named table (a DataFrame) that later cells can use.	Getting the numbers out of v_loans.
Python cell	Runs Python code. It can read the tables produced by earlier cells.	Statistics, clustering, modelling and the Plotly charts.
Chart cell	Draws a chart from a table using point-and-click settings.	The bar charts and the line chart in Tasks 1 and 2.
Markdown (text) cell	Holds written explanations with headings, bullets and tables.	The notes under each task and the top-of-notebook limitations note.

5.2 Connecting Hex to BigQuery

Hex needs its own "login" to read from BigQuery. We created a **service account**, a robot identity, named hex-reader, and gave it the smallest set of permissions that work. Its key file (a small JSON text file that acts like a password) was pasted into Hex's connection form and is stored outside the project folder and never published.

Role (permission bundle)	What it allows	Why it is needed
BigQuery Data Viewer	Read the tables and the view.	So that queries can return data.
BigQuery Job User	Run queries.	Without it, reading is allowed but starting a query is refused (error: bigquery.jobs.create denied).
BigQuery Read Session User	Open a fast read channel (the Storage API).	Hex uses this faster channel to pull large results into Python. Missing at first: the error "bigquery.readsessions.create denied" appeared, and adding this role fixed it.

None of the roles allows changing or deleting data: Hex can only **read**. The connection is named pave-bank-bigquery. A connection test query (count of rows in v_loans) returned 270,299, the same number as in BigQuery, confirming everything was wired correctly.

5.3 How the notebook is organised

The notebook follows the order of the assignment. It starts with a note explaining the dataset's limitations, then has one section per task. Each section repeats the same rhythm: a SQL cell gets the numbers, a chart or Python cell shows them, and a Markdown cell explains what they mean.

Section	Cells, in order
Top note	Markdown: data limitations and how we adapted (Section 11).
Task 1	SQL (counts by year and outcome) - bar chart - Markdown (definitions) - SQL (percentages) - grouped bar chart - Markdown (findings).
Task 2	SQL (yearly summary table) - SQL (long format for the chart) - line chart - Markdown (findings).
Task 3	SQL (what late looks like) - SQL (list of late customers) - SQL (late rate by rate band) - Python bubble chart - Markdown (findings).
Task 4	SQL (customer table) - installation of scikit-learn - Python (prepare features) - report table - Python (choose k) - k chart - Python (fit and profile) - bubble chart - Markdown.
Task 5	Python (rates by characteristic) - table - Python (predictive model) - two tables - Python (lending rules) - Markdown (insights and recommendations).

5.4 How cells pass data to each other

Each SQL cell is given a **result variable name**; that name then works as a Python DataFrame in any later cell. This is how the notebook stays connected without copying data by hand.

Result variable	Produced by	Used by
task1_loans_by_year_status	SQL: counts by year and outcome	Chart cell 1
task1_pct_by_year_status	SQL: percentage share by year	Chart cell 2
task2_performance_by_year	SQL: yearly averages and default rates	Read as a table
task2_default_rate_long	SQL: both default rates, one row per year and measure	Line chart
task3_late_overview / task3_late_customers	SQL: late statuses and the list of late loans	Read as tables
task3_late_by_rate	SQL: late rate per rate band and term	Python bubble chart
task4_customers	SQL: one row per customer with all columns	Every Python cell in Tasks 4 and 5
X, X_scaled, report	Python: feature preparation	Clustering (X_scaled), model (X)
k_report, profile	Python: k comparison, segment summary	k chart, segment bubble chart
drivers, ranking, default_table, late_table, model_summary, rules_table	Python: Task 5 analyses	Read as tables

5.5 Problems met in Hex, and how they were solved

Problem	Cause	Solution
Hex could not connect: readsessions.create permission denied	The service account lacked the Read Session User role.	Added the role in Google Cloud (IAM).
ModuleNotFoundError: No module named 'sklearn'	scikit-learn was not installed in the notebook's Python. The first install (!pip) went to a different Python.	Installed into the notebook's own environment with: !pip --python /ipython/.venv/bin/python install scikit-learn.
Hex's built-in scatter chart cannot size bubbles by a column	Feature missing in that chart type.	Built the bubble charts in Python with Plotly, which supports it.
A native scatter showed nonsense values	The colour column (term) was being summed, which merged both terms and added percentages together.	Removed that chart and used the Python bubble chart, which does not aggregate.
Only the first prints of a cell showed	Hex reliably shows only the final expression of a cell.	Each cell ends with the object to display, and tables are shown in their own cell.
Numbers shifted by 0.1-0.2 points between runs (rate quintiles)	Loans with identical values were split into buckets in row order, and BigQuery returns rows in a changing order.	Sort by loan_id before bucketing, so results are identical on every run (tested with shuffled data).

6. Task 1: Loan outcomes by year

The task in one sentence

Count the loans by outcome (active, paid off, defaulted) and by the year they were created, and show them as a bar chart.

6.1 The approach

Every loan has a status and an origination year. Grouping the loans by these two columns and counting gives a small table of 8 years times 3 outcome groups = 24 numbers. The groups were defined once in the view:

paid_off means Fully Paid; **defaulted** means Charged Off or Default; **active** means the loan is still open (Current, in a grace period, or late).

6.2 Query 1: how many loans per year and outcome

Hex SQL cell, result variable `task1_loans_by_year_status`

```
SELECT
  issue_year,
  status_group,
  COUNT(*) AS loans          -- amount of loans for the period
FROM `pave-bank-fintech.fintech_test.v_loans`
GROUP BY issue_year, status_group
ORDER BY issue_year, status_group;
```

What this calculates

The number of loans for each combination of origination year and outcome group. There are 8 years and 3 groups, so the result has 24 rows.

How it works, step by step

- **FROM v_loans** reads the cleaned view (one row per loan).
- **GROUP BY issue_year, status_group** puts all loans with the same year and the same group together, so that each pair becomes one result row.
- **COUNT(*) AS loans** counts the rows in each group. Because each row is a loan, this is the number of loans. The name after AS is the label of the new column.
- **ORDER BY issue_year, status_group** sorts the output: by year, then alphabetically by outcome.
- Rule of SQL: every column in SELECT must be either in the GROUP BY or wrapped in a counting/summing function such as COUNT.

What the result shows

The 24 counts add up to 270,299, the full dataset. For example, the three 2016 rows are active 37,331, defaulted 1,122 and paid_off 4,915, which add up to 43,368, the total loans of 2016.

The idea to deliver

This is the raw material of the chart: how many loans each year ended in each outcome.

Year	active	paid_off	defaulted	total
2012	112	2,094	388	2,594
2013	1,168	10,334	1,958	13,460
2014	8,214	11,823	3,416	23,453
2015	4,208	30,085	7,626	41,919
2016	37,331	4,915	1,122	43,368
2017	32,177	9,831	2,427	44,435
2018	45,674	3,340	319	49,333

Year	active	paid_off	defaulted	total
2019	47,191	3,939	607	51,737

6.3 Chart 1: loans by outcome and origination year

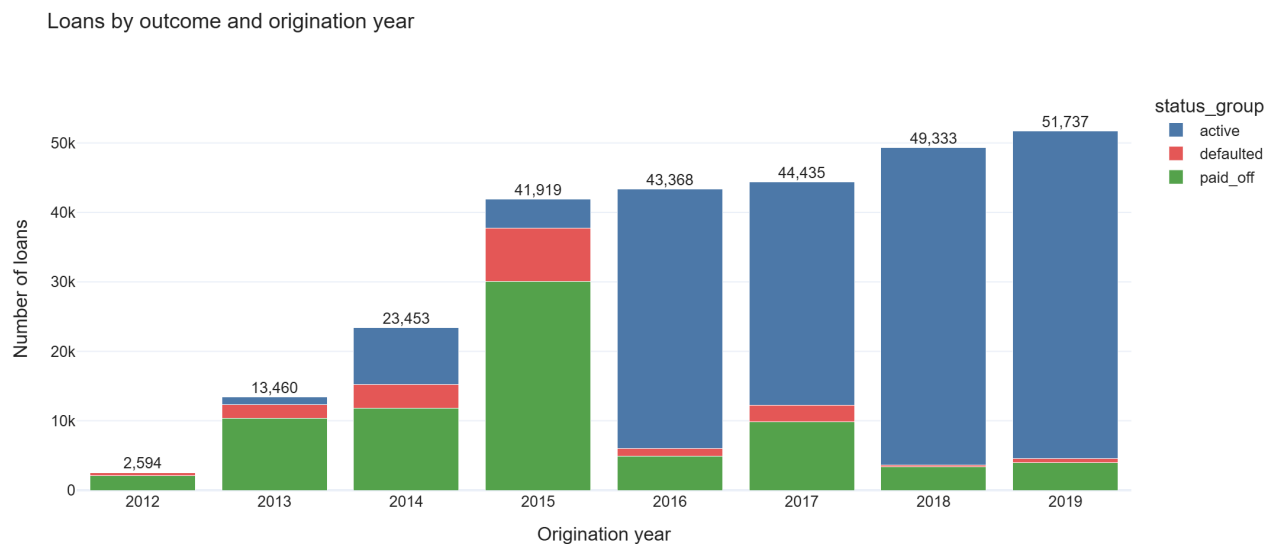


Chart 1 (stacked bar chart). Each bar is one origination year; its height is the number of loans, split by outcome. Re-created from the same data as the Hex chart cell.

What the chart shows

For each year from 2012 to 2019, one bar whose total height is the number of loans originated that year, colour-coded by outcome: green for paid off, red for defaulted and blue for still active.

How to read it

- Height of the whole bar = total loans that year. The number above each bar states it: 2,594 in 2012 growing to 51,737 in 2019.
- Green (paid off) at the bottom, red (defaulted) in the middle, blue (active) on top.
- Older years (2012-2015) are mostly green and red because those loans have had time to finish. Newer years (2016-2019) are mostly blue because most loans are still running.

The idea to deliver

Loan volume grew about twenty-fold in eight years, and the outcome mix depends strongly on how old the loans are. Note the oddly large blue share for 2016: it is investigated on the next chart.

6.4 Query 2: the same data as percentages

Counts hide proportions: the red slivers in the counts chart are hard to compare. The second query divides each count by the year's total.

Hex SQL cell, result variable task1_pct_by_year_status

```
SELECT
  issue_year,
  status_group,
  COUNT(*) AS loans,
  ROUND(100 * COUNT(*) / SUM(COUNT(*)) OVER (PARTITION BY issue_year), 2) AS pct_of_year -- each status share (%) of its year's total
  » loans, rounded to 2 decimal places
FROM `pave-bank-fintech.fintech_test.v_loans`
GROUP BY issue_year, status_group
ORDER BY issue_year, status_group;
```

What this calculates

Each outcome's share of its year's loans, in percent: $\text{share} = 100 \times (\text{loans with that outcome in the year}) / (\text{all loans in that year})$.

How it works, step by step

- **COUNT(*)** is the loan count of each year-and-outcome pair, as before.
- **SUM(COUNT(*)) OVER (PARTITION BY issue_year)** is a **window function**: it adds up the counts of all rows that share the same year, without collapsing the rows. For the three 2016 rows that sum is $37,331 + 1,122 + 4,915 = 43,368$, and every 2016 row receives that same total next to its own count.
- **100 * COUNT(*) / SUM(...)** divides the row's count by its year total and multiplies by 100. Worked example, 2016 defaulted: $100 \times 1,122 / 43,368 = 2.59$.
- **ROUND(..., 2)** keeps two decimals: 2.59.
- A plain SUM with GROUP BY would merge the rows; a window function keeps every row and adds the year total beside it, which is what division needs.

What the result shows

For every year the three percentages add up to 100. For 2016: active 86.08%, defaulted 2.59%, paid_off 11.33%.

The idea to deliver

Percentages make different years comparable regardless of how many loans each had.

Year	active %	paid_off %	defaulted %
2012	4.32	80.72	14.96
2013	8.68	76.78	14.55
2014	35.02	50.41	14.57
2015	10.04	71.77	18.19
2016	86.08	11.33	2.59
2017	72.41	22.12	5.46
2018	92.58	6.77	0.65
2019	91.21	7.61	1.17

6.5 Chart 2: the mix of outcomes in each year

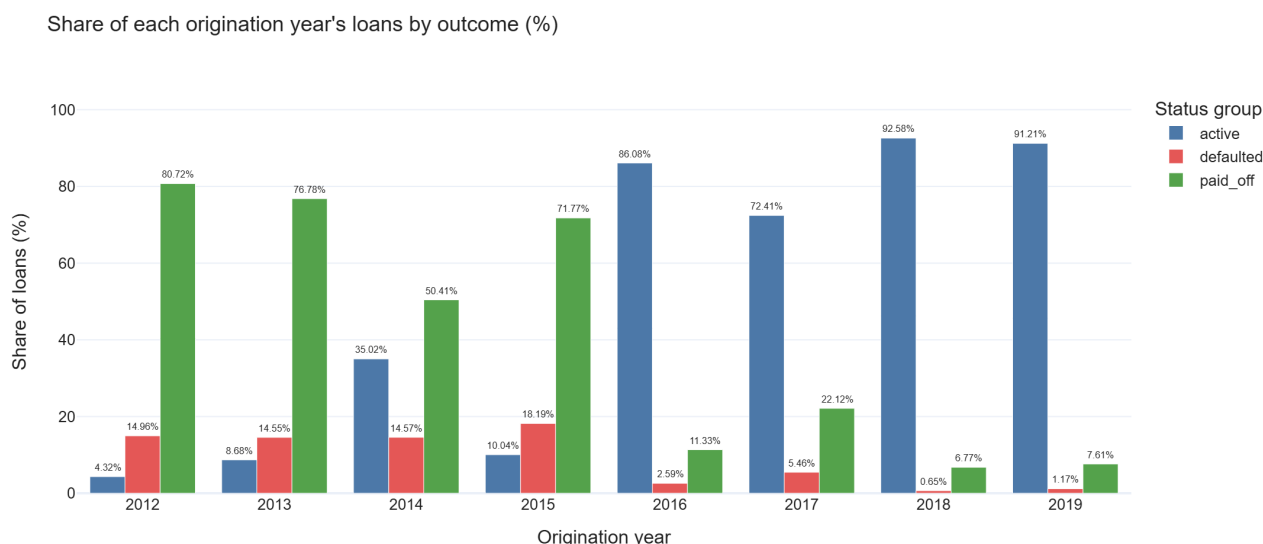


Chart 2 (grouped bar chart). For each year, three bars show the share of loans that are active (blue), defaulted (red) and paid off (green).

What the chart shows

For each origination year, three side-by-side bars: the percentage of that year's loans that are active, defaulted and paid off. The three bars of a year add up to 100%.

How to read it

- Look at the red bars: 2012-2015 sit between 14.5% and 18.2%; 2016 collapses to 2.6%; 2017 is 5.5%; 2018 and 2019 are under 2%.
- Look at the blue bars: 2016 is 86.1% and 2018-2019 are above 91%.
- Look at the green bars: paid-off share falls from about 81% (2012) to 7.6% (2019) simply because recent loans are still being repaid.

The idea to deliver

Three findings. First, older loans defaulted at roughly one in six. Second, recent loans look safe only because they are young (many have not had time to default). Third, 2016 is an anomaly: 86% of its loans are still open although a 36-month loan from 2016 should be finished by 2019, for both 36- and 60-month loans, which suggests the statuses were recorded at different dates for different years.

Caveat to state openly

The 2016 pattern is a fact about the data; the explanation (different snapshot dates) is our hypothesis and is not proven. Because of it, year-to-year default counts must be compared carefully. Task 2 handles this by measuring the default rate two ways.

7. Task 2: Loan performance by year

The task in one sentence

Compute, for each year, the average interest rate, the average principal and the default rate, and draw the default-rate trend as a line chart.

7.1 The approach and the key decision

Averages are straightforward. The **default rate** needs care, because its meaning depends on what you divide by. Two versions were computed:

- **Default rate against all loans** = defaulted loans / all loans of that year x 100. This is the standard definition, but it has a flaw for recent years: loans that are still open cannot have defaulted yet, so they drag the rate down.
- **Default rate against resolved loans only** = defaulted loans / (paid-off loans + defaulted loans) x 100. It leaves out open loans, so it is less distorted by loan age. It is not perfect either (for the newest years the few resolved loans are an unrepresentative early slice), but it is a fairer yardstick for comparing years.

7.2 Query 1: the yearly summary table

Hex SQL cell, result variable `task2_performance_by_year`

```
-- Task 2: loan performance by origination year.
-- Returns one row per year with the number of loans, average interest rate,
-- average principal, and the default rate measured two ways
-- (against all loans, and against resolved loans only).

SELECT
  issue_year,
  COUNT(*)
  ROUND(AVG(int_rate) * 100, 2)
    » fraction to a percentage
  ROUND(AVG(loan_amount), 2)
    » dollars
  COUNTIF(is_default)
    » in Default
  COUNTIF(is_resolved)
    » (paid off or defaulted)
  ROUND(100 * COUNTIF(is_default) / COUNT(*), 2)
    » low for recent years: open loans can't default yet)
  ROUND(100 * COUNTIF(is_default) / COUNTIF(is_resolved), 2)
    » (ignores loans still open)
FROM `pave-bank-fintech.fintech_test.v_loans` -- cleaned view, one row per loan
GROUP BY issue_year -- one result row per origination year
ORDER BY issue_year;
```

What this calculates

For each origination year: the number of loans, the average interest rate in percent, the average principal in dollars, the number of defaulted and resolved loans, and both default rates.

How it works, step by step

- **AVG(int_rate) * 100**: the average of the interest-rate column, multiplied by 100 because the rate is stored as a fraction (0.1410 becomes 14.10%). Example: 2012 average 0.1410 = 14.10%.
- **AVG(loan_amount)**: the average amount borrowed, called the principal. 2012: \$13,672.93.
- **COUNTIF(is_default)**: counts only the rows where the true/false column is true, i.e. the defaulted loans. **COUNTIF(is_resolved)** counts loans with a known outcome.
- **default_rate_all_pct** = 100 x COUNTIF(is_default) / COUNT(*). Worked example 2016: 100 x 1,122 / 43,368 = 2.59.
- **default_rate_resolved_pct** = 100 x COUNTIF(is_default) / COUNTIF(is_resolved). Worked example 2016: 100 x 1,122 / 6,037 = 18.59. Here 6,037 is the paid-off loans (4,915) plus the defaulted ones (1,122).
- **ROUND(x, 2)** keeps two decimals. **GROUP BY issue_year** produces one row per year.

What the result shows

Eight rows, one per year. The table below is the result.

The idea to deliver

One table holds the whole performance story: the price of the loans (rate), their size (principal) and how often they fail, measured two ways.

Year	Loans	Avg rate %	Avg principal \$	Defaulted	Resolved	Default % (all loans)	Default % (resolved)
2012	2,594.0	14.10	13,672.93	388.0	2,482.0	14.96	15.63
2013	13,460.0	14.54	14,811.50	1,958.0	12,292.0	14.55	15.93
2014	23,453.0	13.81	14,849.83	3,416.0	15,239.0	14.57	22.42
2015	41,919.0	12.60	15,254.05	7,626.0	37,711.0	18.19	20.22
2016	43,368.0	13.04	14,725.87	1,122.0	6,037.0	2.59	18.59
2017	44,435.0	13.25	14,868.28	2,427.0	12,258.0	5.46	19.80
2018	49,333.0	12.75	16,051.65	319.0	3,659.0	0.65	8.72
2019	51,737.0	12.89	16,474.77	607.0	4,546.0	1.17	13.35

- **Averages are stable.** The average rate stays between 12.60% (2015) and 14.54% (2013), and the average principal between about \$13,673 (2012) and \$16,475 (2019). Neither explains the large swings in default rates.
- The all-loans default rate goes 14.96, 14.55, 14.57, 18.19, then **2.59**, 5.46, 0.65 and 1.17. The resolved-only rate goes 15.63, 15.93, 22.42, 20.22, 18.59, 19.80, 8.72 and 13.35.

7.3 Query 2: the same rates in "long" format, for the chart

A line chart draws one line per category. Our two default rates sit in two columns of the same row, so they are restacked into rows (one row per year and per measure).

Hex SQL cell, result variable task2_default_rate_long

```
-- Task 2 (chart data): default rate by origination year, measured two ways.
-- The two rates are stacked on top of each other (UNION ALL) into "long" format:
-- one row per year per measure, which lets Hex draw one line per measure.

-- Part 1: default rate against ALL loans
SELECT issue_year,
       'All loans' AS measure,
       ROUND(100 * COUNTIF(is_default) / COUNT(*), 2) AS default_rate_pct
FROM `pave-bank-fintech.fintech_test.v_loans`
GROUP BY issue_year

UNION ALL
  » every row)

-- Part 2: default rate against RESOLVED loans only
SELECT issue_year,
       'Resolved loans only' AS measure,
       ROUND(100 * COUNTIF(is_resolved) / COUNTIF(is_resolved), 2) AS default_rate_pct
FROM `pave-bank-fintech.fintech_test.v_loans`
GROUP BY issue_year

ORDER BY issue_year, measure;
  » measure
```

What this calculates

The two default rates of each year as separate rows: sixteen rows, with a column called measure saying which rate the row holds.

How it works, step by step

- The first SELECT computes the all-loans rate and adds a fixed text column, 'All loans' AS measure, that labels every row of this part.
- The second SELECT computes the resolved-loans rate and labels it 'Resolved loans only'.

- **UNION ALL** stacks the rows of the second SELECT beneath the first. Both parts must return the same columns in the same order. (UNION without ALL would also remove duplicate rows, which we do not want.)
- The closing ORDER BY applies to the combined result, sorting by year and then by measure.

What the result shows

Sixteen rows: 8 years x 2 measures.

The idea to deliver

The data is now in the shape a line chart needs: one line per measure.

7.4 Chart 3: default rate by origination year, two definitions

Default rate by origination year: two definitions

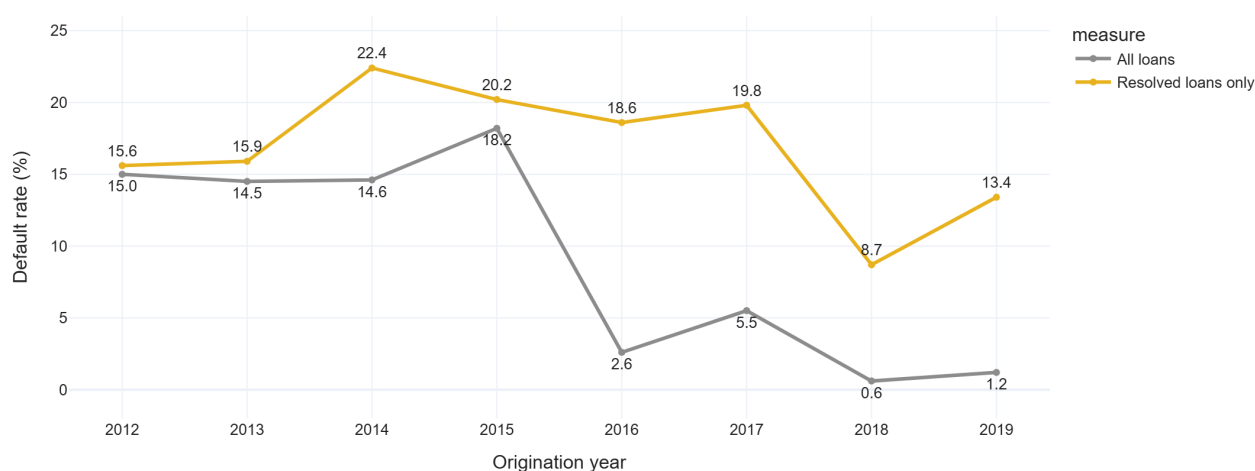


Chart 3 (line chart). Grey: default rate against all loans. Yellow: default rate against resolved loans only. Labels show one decimal.

What the chart shows

Two lines across the years 2012 to 2019. The grey line divides defaulted loans by all loans of the year; the yellow line divides them by the resolved loans only.

How to read it

- The lines are close in 2012-2013 (15.0 versus 15.6, and 14.5 versus 15.9) because those loans are almost all resolved.
- From 2016 the grey line collapses (2.6%, then 5.5%, 0.6%, 1.2%) while the yellow line stays high (18.6%, 19.8%) and only falls at the very end (8.7%, 13.4%).
- The vertical gap between the lines is the share of loans not yet resolved: 0.6 points in 2012 and 16.0 points in 2016.

The idea to deliver

The apparent 'improvement' of credit quality after 2015 is an illusion caused by loans not having finished yet. On the fairer resolved-only basis, default stays around 16-22% for 2012-2017.

What to say about the last two points

The yellow line's 8.7% (2018) and 13.4% (2019) rest on few resolved loans (3,659 and 4,546, compared with 37,711 for 2015) that resolved early, so they are not a reliable sign of better lending. 2014 is also unusual: it has the highest resolved-only rate (22.4%) and 29% of its 36-month loans are still open. We cannot explain it from the data and we say so.

8. Task 3: Payment behaviour

The task in one sentence

Join loans and payments to find customers with late payments, and show late-payment frequency against interest rate as a scatter plot or heatmap.

8.1 What could not be done, and what we did instead

There is **no payments table**, so the requested join between loans and payments is impossible, and there are no payment dates or days-late figures. We agreed a workaround: use the loan's **current status** as the signal of lateness. A loan is called **late** if its status is In Grace Period, Late (16-30 days) or Late (31-120 days). This is a **snapshot** (where the loan stands at one moment), not a payment history: a borrower who was late last year and has since caught up is not counted.

Two more decisions follow from the data. First, each customer has exactly one loan, so "frequency" cannot be a count of late payments per person; it becomes the **share of loans that are late within a group**. Second, only open loans can be late, so the base for the late rate is the **active loans**: $\text{late rate} = \text{late loans} / \text{active loans} \times 100$. Dividing by all loans would make groups with many finished loans look artificially safe.

8.2 Query 1: what "late" looks like among active loans

Hex SQL cell, result variable task3_late_overview

```
-- Task 3, step 1: what "late" looks like among ACTIVE loans.
-- Counts loans by their exact status, marks which statuses count as late (is_late),
-- and shows each status's share of all active loans.

SELECT
  loan_status,                                -- exact status text (Current, In Grace Period, Late (16-30 days)
    » ...)
  is_late,                                    -- true if the status counts as late (our Task 3 definition)
  COUNT(*) AS loans,                         -- number of active loans with this status
  ROUND(100 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct_of_active -- this status's % of all active loans (OVER () = the whole result
    » is one group)
FROM `pave-bank-fintech.fintech_test.v_loans` -- cleaned view, one row per loan
WHERE status_group = 'active'                -- keep only open loans (Current + the three late statuses); only
    » these can be late
GROUP BY loan_status, is_late                -- one row per status
ORDER BY loans DESC;                        -- largest group first
```

What this calculates

A breakdown of the 176,075 active loans by exact status, flagging which statuses count as late, and each status's share of all active loans.

How it works, step by step

- **WHERE status_group = 'active'** filters first: only open loans (Current and the three late statuses) remain. The filter runs before the grouping, so all percentages are measured against active loans.
- **GROUP BY loan_status, is_late** gives one row per status; **is_late** is in the GROUP BY only because it is displayed.
- **SUM(COUNT(*)) OVER ()**: a window function with empty brackets sums the counts of the whole result (176,075). Dividing each count by it gives the share: $100 \times 3,174 / 176,075 = 1.80\%$.
- **ORDER BY loans DESC** puts the biggest group first.

What the result shows

Four rows: Current 170,461 (96.81%); Late 31-120 days 3,174 (1.80%); In Grace Period 1,638 (0.93%); Late 16-30 days 802 (0.46%). The three late rows add up to 5,614, which is 3.19% of active loans.

The idea to deliver

Late payment is rare (about 3 in 100 active loans), and most of it is serious (31-120 days late).

8.3 Query 2: the customers with late payments

Hex SQL cell, result variable task3_late_customers

```
-- Task 3, step 2: the customers with late payments.
-- One row per late loan (= one customer, since each customer has one loan),
-- with the loan's and borrower's main characteristics.
-- Sorted most severe first, then largest loan first.

SELECT
  loan_id,                -- readable ID for the loan/customer (customer_id itself is an unreadable hash)
  loan_status,            -- how late: Late (31-120 days), Late (16-30 days) or In Grace Period
  loan_amount,            -- loan size in dollars
  ROUND(int_rate * 100, 2) AS int_rate_pct, -- interest rate converted from a fraction to a percentage (0.0799 -> 7.99)
  grade,                  -- the lender's risk grade, A (best) to G (worst)
  term_months,            -- loan length: 36 or 60 months
  issue_year,             -- year the loan was originated
  application_type,       -- Individual, Joint or Direct Pay
  home_ownership,         -- MORTGAGE, RENT, OWN, ...
  annual_inc,             -- borrower's annual income
  ROUND(loan_to_income, 3) AS loan_to_income, -- loan amount divided by annual income, rounded to 3 decimals
  state                  -- US state
FROM `pave-bank-fintech.fintech_test.v_loans`
WHERE is_late
ORDER BY
  CASE loan_status
    WHEN 'Late (31-120 days)' THEN 1
    WHEN 'Late (16-30 days)' THEN 2
    ELSE 3
  END,
  loan_amount DESC,
  loan_id;                -- cleaned view, one row per loan
                          -- keep only late loans (true/false column, so no "= true" needed)
                          -- custom sort order: severity, not alphabetical
                          -- most severe first
                          -- everything else late = In Grace Period, last
                          -- within each severity, largest loans first
```

What this calculates

The list of all customers with a late loan: 5,614 rows, one per customer because each has one loan, with the loan's and borrower's main characteristics. This answers the brief's request to "find customers with late payments".

How it works, step by step

- **WHERE is_late** keeps only late loans. `is_late` is already true/false, so no comparison is needed.
- **ROUND(int_rate * 100, 2)** shows the rate as a percentage (0.0799 becomes 7.99). **loan_to_income** is the loan amount divided by annual income, rounded to three decimals.
- **ORDER BY CASE ...**: a CASE turns each status into a number (1 = Late 31-120 days, 2 = Late 16-30 days, 3 = In Grace Period) used only for sorting, so the most severe are listed first. Ties are broken by larger loans first and then by `loan_id`, so the order is the same every time.
- `loan_id` is the readable handle for each customer because `customer_id` is an unreadable hash.

What the result shows

5,614 rows. The first rows are all 31-120 days late with the largest loans (\$40,000).

The idea to deliver

The customers at risk are identified, with the facts a collections team would need. Caution: the top of this list looks like prime borrowers (grade A, incomes of \$100,000 or more) only because it is sorted by loan size, not by risk.

loan_id	Status	Amount \$	Rate %	Grade	Term	Year	Loan-to-income
264364	Late (31-120 days)	40,000	7.99	A	36	2016	0.160
264391	Late (31-120 days)	40,000	13.49	C	36	2016	0.364
264445	Late (31-120 days)	40,000	7.21	A	36	2018	0.308
264818	Late (31-120 days)	40,000	6.46	A	36	2019	0.178
264991	Late (31-120 days)	40,000	6.46	A	36	2018	0.296
265208	Late (31-120 days)	40,000	6.46	A	36	2019	0.364

8.4 Query 3: late rate by interest-rate band and loan term

Hex SQL cell, result variable task3_late_by_rate

```
-- Task 3, step 3: late-payment rate by interest-rate band and loan term.
-- Loans are grouped into 2-point interest-rate bands (4-6%, 6-8%, ...) and by term (36/60 months);
-- for each group it counts active loans, how many are late, and the late rate.
-- Feeds the Task 3 bubble chart.

-- Step A: keep only active loans and compute each loan's rate band
WITH active AS (
  SELECT
    -- Rate band: convert the rate to a percentage (rounded to 2 decimals to avoid floating-point noise),
    -- divide by 2, round DOWN to a whole number, multiply by 2 -> lower edge of a 2-point band.
    -- Example: 0.1349 -> 13.49 -> 6.745 -> 6 -> 12, so this loan is in the "12-14%" band.
    CAST(FLOOR(ROUND(int_rate * 100, 2) / 2) * 2 AS INT64) AS band_start_pct,
    term_months, -- 36 or 60
    is_late -- true if the loan counts as late
  FROM `pave-bank-fintech.fintech_test.v_loans`
  WHERE status_group = 'active'
)

-- Step B: count and compare within each band and term
SELECT
  band_start_pct, -- lower edge of the band (4, 6, 8, ... 30)
  CONCAT(CAST(band_start_pct AS STRING), '-', CAST(band_start_pct + 2 AS STRING), '%') AS rate_band, -- readable label, e.g. "12-14%"
  term_months, -- loan term
  COUNT(*) AS active_loans, -- active loans in this band and term
  COUNTIF(is_late) AS late_loans, -- how many of them are late
  ROUND(100 * COUNTIF(is_late) / COUNT(*), 2) AS late_rate_pct -- late loans as a % of active loans in the group
FROM active
GROUP BY band_start_pct, term_months
ORDER BY band_start_pct, term_months;
-- one row per band and term
```

What this calculates

The late rate for each combination of a two-point interest-rate band (4-6%, 6-8%, ... 30-32%) and loan term (36 or 60 months): 14 bands x 2 terms = 28 rows. This is the data for the chart.

How it works, step by step

- **WITH active AS (...)** defines a temporary named result (a common table expression). Step A keeps only active loans and labels each with its band; step B then counts.
- **The band arithmetic:** `ROUND(int_rate * 100, 2)` turns the fraction into a percentage; dividing by 2 and `FLOOR` (round down) gives the number of complete two-point steps; multiplying by 2 gives the lower edge of the band. Worked example: rate 0.1349 gives 13.49; $13.49 / 2 = 6.745$; rounded down 6; $6 \times 2 = 12$; so the loan is in the 12-14% band. The inner `ROUND` protects against computer arithmetic noise (0.29×100 can come out as 28.999999999999996 and land in the wrong band).
- **CONCAT(...)** joins text pieces into a label such as "12-14%"; `CAST(... AS STRING)` converts numbers to text first.
- **COUNT(*)** = active loans in the group; **COUNTIF(is_late)** = late loans; **late rate = 100 x late / active**. Example: band 12-14%, 36 months: 527 late of 14,968 active = 3.52%.
- Splitting by term matters because 60-month loans have higher rates and different behaviour; mixing them could create a false relationship.

What the result shows

28 rows. The active loans add up to 176,075 and the late loans to 5,614. The full result is shown below.

The idea to deliver

Late-payment frequency measured against interest rate, in the exact form the brief asked for.

Rate band	Term (months)	Active loans	Late loans	Late rate %
4-6%	36	4,004	26	0.65
4-6%	60	49	0	0.00
6-8%	36	25,182	262	1.04
6-8%	60	1,611	11	0.68
8-10%	36	17,295	301	1.74
8-10%	60	5,286	76	1.44
10-12%	36	24,546	673	2.74
10-12%	60	8,749	151	1.73

Rate band	Term (months)	Active loans	Late loans	Late rate %
12-14%	36	14,968	527	3.52
12-14%	60	11,096	273	2.46
14-16%	36	10,884	504	4.63
14-16%	60	9,315	309	3.32
16-18%	36	9,879	435	4.40
16-18%	60	8,083	320	3.96
18-20%	36	4,419	273	6.18
18-20%	60	4,507	263	5.84
20-22%	36	3,044	202	6.64
20-22%	60	3,632	213	5.86
22-24%	36	1,913	130	6.80
22-24%	60	1,894	143	7.55
24-26%	36	1,283	105	8.18
24-26%	60	1,479	130	8.79
26-28%	36	360	39	10.83
26-28%	60	872	80	9.17
28-30%	36	303	15	4.95
28-30%	60	864	70	8.10
30-32%	36	102	18	17.65
30-32%	60	456	65	14.25

8.5 Chart 4: late rate against interest rate, as bubbles

Hex Python cell

```
# Task 3 bubble chart: late-payment rate vs interest rate, split by loan term.
# Each bubble is one interest-rate band for one term; bubble size = number of active loans behind it.
# Input: task3_late_by_rate, the result of the Hex SQL cell (28 rows: 14 rate bands x 2 terms).

import plotly.express as px # Plotly Express: high-level library for interactive charts

df = task3_late_by_rate.copy() # work on a copy so the original SQL
    » result stays unchanged # 36 -> "36 months" (text, so it is
df["term"] = df["term_months"].astype(int).astype(str) + " months" # treated as a category/colour, not a number)
df["band_mid_pct"] = df["band_start_pct"] + 1 # centre of each 2-point band (the 4-6%
    » band is plotted at 5)

fig = px.scatter(
    df,
    x="band_mid_pct",
    y="late_rate_pct",
    » late
    size="active_loans", # bubble size: number of active loans
    » in the group (bigger = more loans)
    color="term",
    color_discrete_map={"36 months": "#7b5cd6", "60 months": "#f28e2b"}, # one colour per loan term
    » for 60 # fixed colours: purple for 36, orange
    category_orders={"term": ["36 months", "60 months"]}, # legend order
    size_max=50, # largest bubble is 50 pixels wide
    hover_name="rate_band", # bold title in the hover box, e.g. "12-14%"
    hover_data={"active_loans": ":", "late_loans": ":", "late_rate_pct": ":.2f", # extra hover fields with formatting
    » (thousands separators, 2 decimals)
    "band_mid_pct": False, "term": False}, # False = hide these fields from the
    » hover box
    labels={"late_rate_pct": "Late rate (%)", "active_loans": "Active loans", # friendly names shown in the hover box
    » and legend
    "late_loans": "Late loans", "term": "Loan term"},
    title="Late-payment rate vs interest rate (active loans; bubble size = number of loans)",
)

# Bubble styling: smallest bubble is 4 px so tiny groups stay visible; slightly see-through; thin white outline
fig.update_traces(marker=dict(sizemin=4, opacity=0.75, line=dict(width=1, color="white")))

# X axis: title, and tick marks every 2 points starting at 5 (5, 7, 9, ...) so each tick sits on a band midpoint
fig.update_xaxes(title="Interest rate band midpoint (%)", tick0=5, dtick=2)

# Y axis: title, and always start at 0 so bar heights are not visually exaggerated
fig.update_yaxes(title="Late rate (% of active loans)", rangemode="tozero")
```

fig # last line of the cell: Hex displays the chart

What this calculates

A chart with one bubble per rate band and term (28 bubbles). Horizontal position is the interest-rate band, vertical position is the late rate, bubble size is the number of active loans behind that point, and colour is the loan term.

How it works, step by step

- `df = task3_late_by_rate.copy()` takes the SQL result as a table and copies it so the original stays untouched.
- `df["term"]` turns 36 into the text "36 months" so that Plotly treats it as a category (two colours) and not as a number (a colour gradient).
- `band_mid_pct = band_start_pct + 1` is the middle of a two-point band, so the 4-6% band is plotted at 5.
- `px.scatter(...)` draws the chart: x, y, size and color say which column controls which visual property; `size_max=50` makes the biggest bubble 50 pixels wide; `hover_data` controls what appears when you hover (thousands separators and two decimals); labels replace column names with readable text.
- `update_traces / update_xaxes / update_yaxes` fine-tune the look: a minimum bubble size so tiny groups stay visible, slight transparency, ticks at every band midpoint, and a vertical axis that starts at zero.
- The last line, `fig`, is what makes Hex display the chart.

What the result shows

The chart below.

The idea to deliver

This is the required "late payment frequency against interest rate" visual, with an honest display of how much data sits behind each point.

Late-payment rate vs interest rate (active loans; bubble size = number of loans)

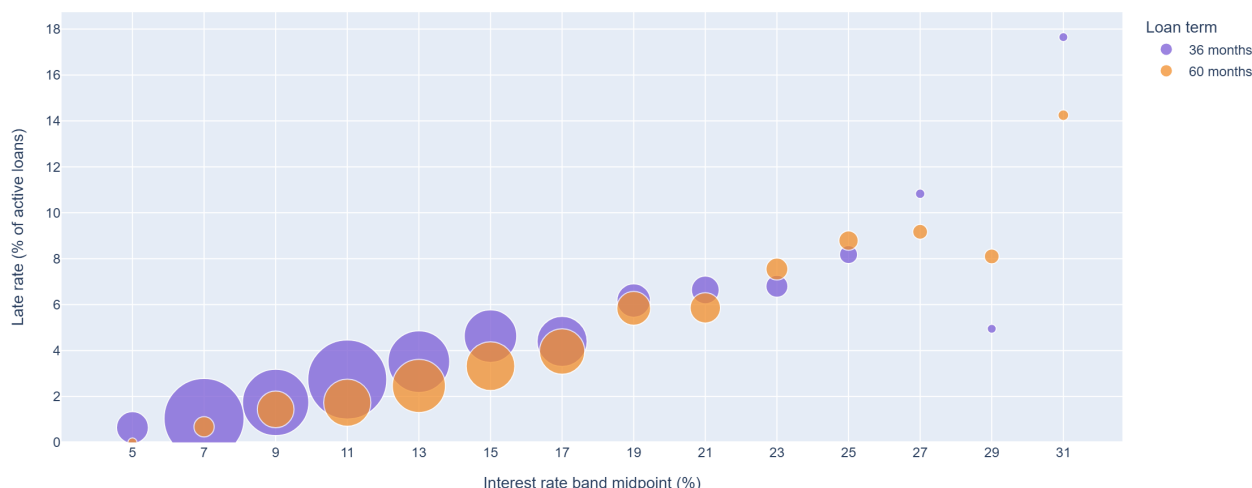


Chart 4 (bubble chart, exactly as produced in the notebook). Purple = 36-month loans, orange = 60-month loans; bubble size = number of active loans.

What the chart shows

Late-payment rate (vertical) against interest-rate band (horizontal), separately for 36-month (purple) and 60-month (orange) loans, with larger bubbles where more loans stand behind the point.

How to read it

- Both colours climb from the lower left to the upper right: from about 0.65% late at the lowest rates to 14-18% at the highest.
- The largest bubbles (6-8% and 10-12% for 36-month loans) sit low: most loans are cheap and rarely late.
- The smallest bubbles at the far right (30-32%: 102 loans at 36 months, 456 at 60 months) are noisy; single points there must not be over-read. The dip at 28-30% for 36-month loans (4.95%) rests on only 303 loans, 15 of them late.
- At the same rate, 36-month loans are late more often than 60-month loans in 11 of the 14 bands. We did not investigate the reason.

The idea to deliver

The higher the interest rate, the more likely a loan is late: across all active loans the late rate rises from 0.6% (4-6% band) to 14.9% (30-32% band), roughly 23 times higher, and the pattern holds within each term. Because the lender sets rates according to the risk it sees, this mostly shows that risk-based pricing works; it does not prove that high rates cause lateness.

9. Task 4: Customer segmentation

The task in one sentence

Use Python to split customers into groups (by loan amount, delay, or credit utilisation) and show the groups as a cluster or bubble chart.

9.1 What could not be done, and what we did instead

Average delay and credit utilisation do not exist in the data (no payment history, no credit limits). We therefore built the groups from what does exist: loan amount, interest rate, term, income, loan-to-income ratio, and a leverage proxy, **balance-to-income = total current balance / annual income** (a borrower with \$150,000 of balances and \$60,000 income scores 2.5). This proxy is **not** credit utilisation, because total balances include mortgages and no credit limits are known.

Design rules: the groups are built *without* looking at outcomes (default or late); outcomes are used only afterwards to describe the groups, so that we can test whether the groups genuinely differ in risk (if we had used outcomes to build the groups, that test would be circular). Each customer has one loan, so "total loan amount" is simply the loan amount.

9.2 Step 1: the customer table

Hex SQL cell, result variable task4_customers

```
-- Task 4: customer-level table for segmentation (the clustering itself happens later in a Python cell).
-- One row per loan = one row per customer (each customer has exactly one loan).
-- Pulls the columns used to build the customer groups, plus columns used only to describe the groups afterwards.

SELECT
  loan_id,                                -- readable ID for each customer (customer_id itself is an unreadable
    » hash)

  -- Clustering inputs (used to build the groups):
  loan_amount,                            -- loan size in dollars
  ROUND(int_rate * 100, 2) AS int_rate_pct, -- interest rate converted from a fraction to a percentage
  term_months,                            -- loan length: 36 or 60 months
  annual_inc,                             -- borrower's annual income
  loan_to_income,                         -- loan amount divided by annual income
  tot_cur_bal,                            -- total current balance across the borrower's accounts (not a clustering
    » input on its own; used for the ratio below)
  SAFE_DIVIDE(tot_cur_bal, annual_inc) AS balance_to_income, -- total balance divided by income: a leverage proxy (returns NULL instead
    » of an error if income is 0)

  -- Description only (NOT used to build the groups):
  grade,                                  -- lender's risk grade, A to G
  application_type,                       -- Individual, Joint or Direct Pay
  home_ownership,                         -- MORTGAGE, RENT, OWN, ...
  issue_year,                             -- year the loan was originated
  status_group,                           -- paid_off, defaulted or active
  is_default,                             -- true if the loan defaulted
  is_late,                                -- true if the loan counts as late
FROM `pave-bank-fintech.fintech_test.v_loans`;
-- cleaned view, one row per loan; no WHERE, so all 270,299 loans are
» returned
```

What this calculates

One row per customer (270,299 rows) with the six columns used to build the groups, and the descriptive columns used afterwards.

How it works, step by step

- **No WHERE and no GROUP BY:** unlike earlier queries, this returns every loan as its own row, because clustering needs individual rows.
- **SAFE_DIVIDE(tot_cur_bal, annual_inc)** is the balance-to-income ratio: total balance divided by income, returning NULL instead of an error if income were zero.
- The columns after the comment "Description only" (grade, application type, home ownership, year, status group, default and late flags) are **not** used to form groups.

What the result shows

270,299 rows and 15 columns; no missing values in the six features.

The idea to deliver

A clean, one-row-per-customer table, with the outcome columns kept aside on purpose.

Hex Python cell: installing scikit-learn into the notebook's own Python environment

```
!pip --python /ipython/.venv/bin/python install scikit-learn
```

The single line above installs the scikit-learn library into the specific Python environment that the notebook uses (the path after `--python`). An earlier attempt with the plain command installed it somewhere the notebook could not see, which is why this explicit form is needed.

9.3 Step 2: preparing the six features

Clustering measures distances between customers. Two problems would spoil that: features on very different scales (income runs to millions, interest rate to about 30), and a few extreme values. Three preparation steps solve them, without removing any customer.

Hex Python cell

```
# Task 4, step 2: prepare the six clustering features.
# Clustering (k-means) measures distances between customers, so features must be on comparable scales
# and free of extreme outliers. Three steps: (1) cap extreme values, (2) log-transform skewed features,
# (3) standardise every feature to mean 0 and standard deviation 1. No customers are removed.
# Output: X_scaled (used for clustering) and a summary table, report.

import numpy as np                                # numerical functions (log, etc.)
import pandas as pd                                # tables (DataFrames)
from sklearn.preprocessing import StandardScaler    # rescales each column to mean 0, std 1

df = task4_customers.copy()                        # SQL result from Hex; a copy keeps the original unchanged
# the six features the clusters are built from
FEATURES = ["loan_amount", "int_rate_pct", "term_months", "annual_inc", "loan_to_income", "balance_to_income"]
# the three features with a long right tail (get a log transform)
SKEWED = ["annual_inc", "loan_to_income", "balance_to_income"]

print("rows:", len(df), "| columns:", df.shape[1])    # number of customers and columns
# total missing cells across the six features
print("NULLs in clustering features:", int(df[FEATURES].isna().sum().sum()))

raw = df[FEATURES].astype(float)                    # the six feature columns, forced to plain decimals

# 1. cap extreme values at the 1st / 99th percentile (rows are kept)
lo, hi = raw.quantile(0.01), raw.quantile(0.99)      # per column: the 1st and 99th percentile values
capped = raw.clip(lower=lo, upper=hi, axis=1)         # pull values below lo up to lo, above hi down to hi
n_capped = ((raw < lo) | (raw > hi)).sum()             # per column: how many values were capped (True = 1)

# 2. log-transform the heavily skewed features
X = capped.copy()                                    # X = capped features; the skewed ones are logged below
for c in SKEWED:                                     # log(1 + x): compresses huge values; the +1 keeps zeros valid
    X[c] = np.log1p(X[c])

# 3. standardise: mean 0, standard deviation 1
# (value - column mean) / column std, for every feature; keep names and row order
X_scaled = pd.DataFrame(StandardScaler().fit_transform(X), columns=FEATURES, index=df.index)

report = pd.DataFrame({                             # summary table to check the preparation worked
    # cut-off values used, and how many customers were capped, per feature
    "cap_low_1%": lo, "cap_high_99%": hi, "values_capped": n_capped,
    "skew_raw": raw.skew(), "skew_prepared": X.skew(),    # lopsidedness before and after (0 = symmetric)
    # should be about 0 and 1 for every feature
    "scaled_mean": X_scaled.mean(), "scaled_std": X_scaled.std(),
}).round(3)                                           # round everything to 3 decimals
print()
print(report.to_string())
```

What this calculates

Six clean, comparable columns called `X_scaled`, and a summary table (report) that proves the preparation worked.

How it works, step by step

- **FEATURES** and **SKEWED** list the six columns to prepare and the three (income, loan-to-income, balance-to-income) with a long tail of very large values.

- **Step 1, capping at the 1st and 99th percentile:** `raw.quantile(0.01)` and `raw.quantile(0.99)` give the values below which 1% and 99% of customers fall. `raw.clip(...)` then pulls anything below the low value up to it and anything above the high value down to it. Example: incomes above \$276,000 are set to \$276,000 and incomes below \$17,000 are set to \$17,000. The row is kept; only the extreme number is softened. `n_capped` counts how many values were changed.
- **Step 2, the logarithm:** `np.log1p(x)` computes $\log(1 + x)$. A logarithm measures proportions instead of absolute gaps: doubling income always adds the same amount (0.69) whether from \$30,000 or from \$150,000. This squeezes the long right tail so that a few very rich customers no longer dominate.
- **Step 3, standardising:** `StandardScaler` applies $z = (\text{value} - \text{mean}) / \text{standard deviation}$ to each column. Example: with mean 50 and standard deviation 10, a value of 70 becomes $(70 - 50) / 10 = 2$. Afterwards every column has mean 0 and spread 1, so no feature counts more merely because its numbers are bigger.
- **The report** shows the cut-off values, how many values were capped, the skewness before and after (a measure of lopsidedness; 0 is symmetric), and a check that the scaled mean is 0 and spread is 1.

What the result shows

The report table below. Skewness fell dramatically: income from 47.19 to 0.07, loan-to-income from 167.99 to 0.74, balance-to-income from 128.11 to 0.34. Capping changed about 2% of the values per feature.

The idea to deliver

The data is fair to every feature and robust to extreme values, and nobody was thrown away.

feature	cap low 1%	cap high 99%	values capped	skew raw	skew prepared	scaled mean	scaled std
loan_amount	1500.000	40000.000	2016	0.776	0.778	0.0	1.0
int_rate_pct	5.320	27.270	3496	0.783	0.692	-0.0	1.0
term_months	36.000	60.000	0	0.884	0.884	0.0	1.0
annual_inc	17000.000	276000.000	5337	47.192	0.068	-0.0	1.0
loan_to_income	0.025	0.732	5256	167.993	0.742	0.0	1.0
balance_to_income	0.056	7.272	5406	128.108	0.342	-0.0	1.0

9.4 Step 3: choosing the number of groups (k)

k-means needs the number of groups (k) in advance. It works like this: place k centres, assign every customer to the nearest centre, move each centre to the average position of its customers, and repeat until nothing changes. To choose k we tried k = 2 to 8 and compared two measures.

Hex Python cell

```
# Task 4, step 3: choose the number of clusters (k) for k-means.
# Fits k-means for k = 2 to 8 on the standardised features (X_scaled) and records two measures per k:
# inertia (falls as k grows; look for an 'elbow') and silhouette (higher = better separated clusters).
# Output: k_report, a 7-row table (one row per k).

import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# tables (DataFrames)
# the k-means clustering algorithm
# measures how well separated the clusters are

rows = []
for k in range(2, 9):
    km = KMeans(n_clusters=k, n_init=5, random_state=42)
    labels = km.fit_predict(X_scaled)
    # silhouette on a random sample of 10,000 customers (all 270k is too slow); -1 to 1, higher = better separated
    sil = silhouette_score(X_scaled, labels, sample_size=10000, random_state=42)
    # store k, inertia (total distance to cluster centres; lower = tighter) and silhouette
    rows.append({"k": k, "inertia": round(km.inertia_), "silhouette": round(sil, 3)})
k_report = pd.DataFrame(rows)
k_report
```

What this calculates

For each k from 2 to 8: the inertia and the silhouette score.

How it works, step by step

- **KMeans(n_clusters=k, n_init=5, random_state=42)**: k groups; n_init=5 means five different random starting positions are tried and the best result kept (a single unlucky start can give a poor grouping); random_state=42 fixes the random choices so the answer is repeatable.
- **fit_predict(X_scaled)** runs the algorithm and returns, for every customer, the number of the group they belong to.
- **Inertia** is the sum, over all customers, of the squared distance to the centre of their group. Smaller means tighter groups, but it always falls as k grows, so we look for an 'elbow' where the fall slows.
- **Silhouette**: for each customer, compare the average distance to their own group with the average distance to the nearest other group. The score runs from -1 to 1; higher means clearer separation; values near 0.2 mean the groups overlap a lot. Computing it for all 270,299 customers is too slow, so silhouette_score uses a random sample of 10,000.

What the result shows

The table below.

The idea to deliver

An evidence-based basis for the choice of k, instead of a guess.

row	k	inertia	silhouette
0	2	1212800	0.269
1	3	1044775	0.202
2	4	925754	0.207
3	5	836670	0.202
4	6	763876	0.199
5	7	717540	0.200
6	8	675593	0.198

Hex Python cell

```
# Task 4, step 3 (chart): compare k = 2..8 visually.
# Left: inertia by k (look for an 'elbow' where the curve flattens). Right: silhouette by k (higher = better).
# Input: k_report from the previous cell.

import plotly.graph_objects as go                                # Plotly's building blocks for charts (traces)
from plotly.subplots import make_subplots                        # puts several charts side by side in one figure

# one figure with two charts side by side (1 row, 2 columns), each with its own title
fig = make_subplots(rows=1, cols=2, subplot_titles=("Inertia (lower = tighter clusters)",
                                                    "Silhouette (higher = better separated)"))

# left chart: inertia against k, drawn as a line with points
fig.add_trace(go.Scatter(x=k_report["k"], y=k_report["inertia"], mode="lines+markers"), row=1, col=1)
# right chart: silhouette against k, drawn as a line with points
fig.add_trace(go.Scatter(x=k_report["k"], y=k_report["silhouette"], mode="lines+markers"), row=1, col=2)
fig.update_xaxes(title="Number of clusters (k)", dtick=1)        # X-axis title on both charts; a tick at every whole k
# no legend (each chart has one line), 380 px tall, overall title
fig.update_layout(showlegend=False, height=380, title="Choosing the number of clusters")
fig
```

What this calculates

A picture of the table: inertia on the left, silhouette on the right, both against k.

How it works, step by step

- **make_subplots(rows=1, cols=2, ...)** creates one figure with two charts side by side.
- Each **add_trace(go.Scatter(...))** draws one line with points; row and col say which of the two charts it belongs to.
- **update_xaxes** and **update_layout** set the axis title, the tick at every whole k, the height and the overall title.

What the result shows

The chart below.

The idea to deliver

Shows visually that there is no clear elbow and that silhouette cannot pick between $k = 3$ and $k = 8$.

Choosing the number of clusters

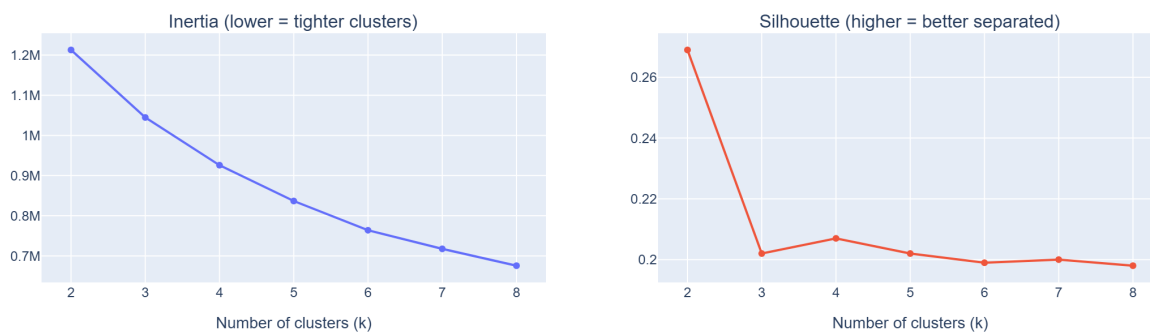


Chart 5 (as produced in the notebook). Left: inertia by k . Right: silhouette by k .

What the chart shows

Two curves. The left curve (inertia) falls smoothly and gradually as k increases. The right curve (silhouette) is highest at $k = 2$ (0.269) and then flat at about 0.20 for $k = 3$ to 8.

How to read it

- No sharp elbow on the left: every extra group helps a bit less than the last (the fall per step shrinks from about 168 thousand to about 42 thousand).
- On the right, values of 0.198 to 0.207 for $k = 3$ to 8 are effectively the same; the differences are noise.
- $k = 2$ scores highest but mostly just separates 36-month from 60-month loans (in a check, one group was 75.5% 60-month loans and the other 0.4%), which we already know.

The idea to deliver

The customers form a continuum rather than naturally separate clumps, so any k is a useful description rather than a law of nature. We chose **$k = 5$** as a judgement call: in a separate check on other random starts it was the most stable choice among $k = 4$ to 7 (similarity between runs 0.993), and unlike $k = 4$ it isolates a group with high balances relative to income. We say plainly that silhouette alone could not decide.

9.5 Step 4: fitting the five groups and describing them

Hex Python cell

```
# Task 4, step 4: fit the final k-means model (k = 5) and profile the five segments.
# Segments are numbered 1-5 by average loan amount (smallest to largest) so the numbering is stable.
# Default and late flags are NOT clustering inputs; they are used only to describe each segment afterwards.
# Output: profile, one row per segment (size, averages/medians, default rate, late rate).

import pandas as pd
from sklearn.cluster import KMeans

# tables (DataFrames)
# the k-means clustering algorithm

km = KMeans(n_clusters=5, n_init=10, random_state=42)
# 5 clusters (chosen in the previous step); 10 random starts; fixed seed =
# repeatable
raw_labels = km.fit_predict(X_scaled)
# fit on all customers; one cluster number (0-4, arbitrary order) per
# customer

seg = task4_customers.copy()
# copy of the customer table to add the cluster labels to
seg["is_default"] = seg["is_default"].astype(float)
# true/false -> 1.0/0.0, so an average becomes a rate
seg["is_late"] = seg["is_late"].astype(float)
# same for the late flag
seg["raw_cluster"] = raw_labels
# attach each customer's cluster number

# cluster numbers ordered from smallest to largest average loan amount
order = seg.groupby("raw_cluster")["loan_amount"].mean().sort_values().index
# map each old cluster number to 1, 2, 3, 4, 5 in that order
relabel = {old: new for new, old in enumerate(order, start=1)}
```

```

seg["segment"] = seg["raw_cluster"].map(relabel)          # final segment number (1 = smallest average loan): stable, readable
    » numbering

resolved = seg["status_group"] != "active"              # loans with a known outcome: paid off or defaulted
active = seg["status_group"] == "active"               # open loans: these can still be late

profile = seg.groupby("segment").agg(                  # one row per segment; each line below defines one column
    customers=("loan_id", "size"),                    # number of customers in the segment
    avg_loan_amount=("loan_amount", "mean"),          # average loan amount
    avg_int_rate_pct=("int_rate_pct", "mean"),        # average interest rate (%)
    # % of the segment's loans that are 60-month (True counts as 1, so the mean is a share)
    pct_60_month=("term_months", lambda s: 100 * (s == 60).mean()),
    median_annual_inc=("annual_inc", "median"),       # median income (the middle value; barely moved by extreme incomes)
    median_loan_to_income=("loan_to_income", "median"), # median loan amount divided by income
    # median total balance divided by income (leverage proxy)
    median_balance_to_income=("balance_to_income", "median"),
)
# insert a column at position 1: each segment's % of all customers
profile.insert(1, "share_pct", 100 * profile["customers"] / len(seg))
# default rate: % of RESOLVED loans that defaulted, per segment
profile["default_rate_resolved_pct"] = 100 * seg[resolved].groupby("segment")["is_default"].mean()
# late rate: % of ACTIVE loans that are late, per segment
profile["late_rate_active_pct"] = 100 * seg[active].groupby("segment")["is_late"].mean()
profile = profile.round(2)                             # round every number to 2 decimals
profile

```

What this calculates

The final grouping (k = 5) and a one-row-per-group summary: size, average loan, average rate, share of 60-month loans, median income, median ratios, default rate and late rate.

How it works, step by step

- KMeans with `n_init=10` (ten starts this time) fits the final five groups; each customer gets a group number 0 to 4.
- The group numbers from k-means are arbitrary and can change between runs, so the groups are **renumbered 1 to 5 by average loan amount** (order, relabel, segment), which makes segment 1 always the smallest-loan group.
- `is_default` and `is_late` are converted from true/false to 1.0/0.0, so that an average becomes a rate: the average of a column of ones and zeros is the share of ones.
- resolved** and **active** select the loans with a known outcome and the open loans.
- .agg(...)** builds the summary: "size" counts rows, "mean" averages, "median" is the middle value (not moved by extreme incomes). The lambda expression `100 * (s == 60).mean()` is the percentage of 60-month loans.
- default_rate_resolved_pct** = 100 x (defaulted loans of the group) / (resolved loans of the group);
late_rate_active_pct = 100 x (late loans of the group) / (active loans of the group). These are the same definitions used in Tasks 2 and 3.

What the result shows

The profile table below.

The idea to deliver

The customer base is described by five understandable types, and outcomes (kept out of the grouping) show whether they truly differ.

segment	customers	share pct	avg loan amount	avg int rate pct	pct 60 month	median annual inc	median loan to income	median balance to income	default rate resolved pct	late rate active pct
1	67155	24.84	8010.5	12.03	0.83	60000.0	0.13	0.48	14.86	2.68
2	52812	19.54	9446.1	11.12	1.68	70000.0	0.13	3.12	10.61	2.48
3	39091	14.46	13829.44	14.14	3.61	37906.0	0.33	0.75	23.24	4.02
4	72961	26.99	20737.41	16.15	99.57	70100.0	0.28	1.74	32.30	3.87
5	38280	14.16	28099.05	10.69	13.13	120000.0	0.23	1.62	12.98	2.76

Hex Python cell

```

# Task 4, step 5: bubble chart of the five customer segments.
# X = average loan amount, Y = default rate on resolved loans, bubble area = number of customers.
# Input: profile from the previous cell (one row per segment).

```

```

import plotly.express as px                                # Plotly Express: high-level library for interactive charts

names = {                                                  # segment number -> descriptive name (chosen from each segment's profile)
    1: "Small loans, low leverage",
    2: "Small loans, high existing balances",
    3: "Low income, stretched",
    4: "60-month, higher-rate loans",
    5: "Large loans, high income",
}
plot_df = profile.reset_index()                            # turn the 'segment' index into a normal column
# legend text such as "4 · 60-month, higher-rate loans" (number + name)
plot_df["label"] = plot_df["segment"].astype(str) + " · " + plot_df["segment"].map(names)

fig = px.scatter(                                          # one bubble per segment (5 bubbles)
    plot_df,                                              # the segment profile table from the previous cell
    x="avg_loan_amount",                                # X axis: average loan amount
    y="default_rate_resolved_pct",                       # Y axis: default rate on resolved loans (%)
    size="customers",                                    # bubble area = number of customers in the segment
    color="label",                                       # one colour per segment (also builds the legend)
    text="segment",                                     # segment number printed inside each bubble
    size_max=60,                                         # largest bubble is 60 pixels wide
    hover_name="label",                                 # bold title of the hover box
    hover_data={                                         # extra fields in the hover box, with number formats
        # formats: ':,' = thousands separator, ':.1f' / ':.2f' = 1 / 2 decimals
        "customers": ":,", "share_pct": ":.1f", "avg_int_rate_pct": ":.2f", "pct_60_month": ":.1f",
        # ':,.0f' = thousands separator, no decimals
        "median_annual_inc": ":,.0f", "late_rate_active_pct": ":.2f", "default_rate_resolved_pct": ":.1f",
        # False = hide (already visible on the axis, legend or inside the bubble)
        "avg_loan_amount": False, "label": False, "segment": False,
    },
    # friendly names shown in the hover box instead of column names
    labels={
        "customers": "Customers", "share_pct": "Share of customers (%)", "avg_int_rate_pct": "Avg interest rate (%)",
        "pct_60_month": "60-month loans (%)", "median_annual_inc": "Median income ($)",
        "late_rate_active_pct": "Late rate, active loans (%)", "default_rate_resolved_pct": "Default rate, resolved (%)",
        "label": "Segment",
    },
    # chart title
    title="Customer segments: loan size vs default rate (bubble size = number of customers)",
)
# bubble styling: number centred in white 15 pt text; slightly see-through bubbles with a thin white outline
fig.update_traces(textposition="middle center", textfont=dict(color="white", size=15),
                  marker=dict(opacity=0.85, line=dict(width=1, color="white")))
# X axis: title, $ prefix and thousands separators on ticks, fixed range 0 to 34,000
fig.update_xaxes(title="Average loan amount ($)", tickprefix="$", tickformat="," , range=[0, 34000])
# Y axis: title and fixed range 0 to 40 (%)
fig.update_yaxes(title="Default rate, resolved loans only (%)", range=[0, 40])
fig.update_layout(legend_title_text="Segment", height=520) # legend heading; chart 520 px tall
fig

```

What this calculates

A bubble chart of the five segments: horizontal position is the average loan amount, vertical position is the default rate on resolved loans, bubble size is the number of customers.

How it works, step by step

- **names** is a dictionary giving each segment number a descriptive name, chosen by reading each segment's profile.
- **profile.reset_index()** turns the segment number from the row label into a normal column, which Plotly needs.
- **plot_df["label"]** builds legend text such as "4 · 60-month, higher-rate loans" from the number and its name.
- **px.scatter** draws one bubble per segment; **hover_data** with format codes (":," thousands separator, ":.1f" one decimal) fills the hover box; **text="segment"** prints the number inside each bubble.
- The axis ranges are fixed (0 to 34,000 dollars and 0 to 40 percent) so all bubbles fit with margin.

What the result shows

The chart below.

The idea to deliver

The segments in one picture, with their size and their risk.

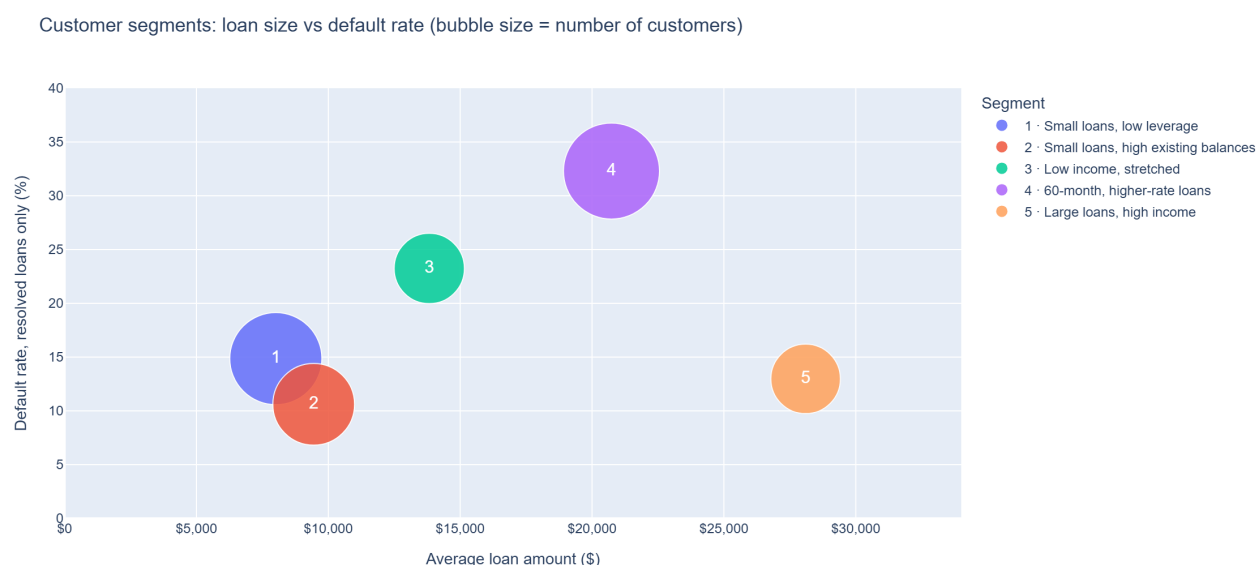


Chart 6 (bubble chart, exactly as produced in the notebook). Each bubble is a customer segment; bubble area = number of customers.

What the chart shows

Five bubbles. The height of a bubble is the default rate among that segment's resolved loans; its horizontal position is the segment's average loan; its size is the number of customers.

How to read it

- Segment 4 (60-month, higher-rate loans; 72,961 customers) is highest at 32.3% default.
- Segment 3 (low income, stretched; 39,091 customers) is next at 23.2%.
- Segments 1, 2 and 5 are the safest: 14.9%, 10.6% and 13.0%.
- Segments 1 and 2 overlap slightly because their average loans are close (\$8.0k and \$9.4k).

The idea to deliver

Five customer types differ up to threefold in default rate (10.6% to 32.3%) even though outcomes were not used to form them, which suggests the groups capture real differences in risk.

Segment	Name	Who they are (from the profile)
1	Small loans, low leverage	24.8% of customers; average loan \$8.0k, 36-month terms, median income \$60k, low loan-to-income (0.13) and low balances (0.48 of income); default 14.9%.
2	Small loans, high existing balances	19.5%; average loan \$9.4k, low rates (11.1%), median income \$70k, balances 3.12 times income (highest); the lowest default (10.6%) and late (2.5%) rates. Balances include mortgages, so high balances may reflect home ownership, not risk.
3	Low income, stretched	14.5%; median income only \$37.9k, highest loan-to-income (0.33); rate 14.1%; default 23.2% and the highest late rate (4.0%).
4	60-month, higher-rate loans	27.0%; 99.6% are 60-month loans, average loan \$20.7k, highest rates (16.2%), default 32.3%. Much of the high default is the known effect of long terms.
5	Large loans, high income	14.2%; average loan \$28.1k, median income \$120k, lowest rates (10.7%); default 13.0%.

Caveats to state

Default rates use resolved loans only and carry the snapshot caveat from Tasks 1-2, so they compare segments; they are not exact risk levels.

The late rates differ much less across segments (2.5% to 4.0%) than default rates do.

K-means can settle slightly differently on another machine: membership shifted by less than 1% of customers in tests, without changing the conclusions.

10. Task 5: Insights and recommendations

The task in one sentence

Explain which customer or loan characteristics best predict default or late payment, and what you would flag or change if you designed lending rules.

10.1 The approach: three ways of looking, so the answer does not rest on one method

Lens	Question it answers	Method
1. Buckets	Does the default rate differ across groups of a characteristic (for example across the five income fifths)?	Group by characteristic, compare rates (Section 10.2).
2. A model	When all characteristics are considered together, which ones still matter, and how good is the prediction?	Logistic regression scored on unseen data (Section 10.3).
3. Rules	If a lender used a simple rule such as "grade E or worse", how many defaults would it catch and at what cost?	Test candidate rules on history (Section 10.4).

If all three lenses point the same way, the finding is credible. They did.

10.2 Lens 1: default and late rates by characteristic

Hex Python cell

```
# Task 5, step 1: which characteristics separate risky loans from safe ones?
# Cuts each characteristic into buckets (numeric ones into five equal-sized groups) and compares the default rate
# (resolved loans) and late rate (active loans) across buckets. Nothing is modelled or removed here.
# Outputs: drivers (one row per characteristic and bucket) and ranking (best-vs-worst bucket spread per characteristic).

import pandas as pd                                # tables (DataFrames)

# sorted by loan_id so tied values always land in the same bucket, whatever order BigQuery returns rows in
d = task4_customers.sort_values("loan_id").reset_index(drop=True)
d["is_default"] = d["is_default"].astype(float)      # true/false -> 1.0/0.0, so an average becomes a rate
d["is_late"] = d["is_late"].astype(float)           # same for the late flag

# ---- buckets (computed on ALL loans so a bucket means the same thing for default and late) ----
QLAB = ["Q1 (lowest)", "Q2", "Q3", "Q4", "Q5 (highest)"] # labels for the five equal-sized groups

def quintile(s):
    return pd.qcut(s.rank(method="first"), 5, labels=QLAB) # cut a column into five equal-sized groups
                                                         # rank first (ties split evenly), then cut into 5

# 36 -> "36 months" (text label)
d["term"] = d["term_months"].astype(int).astype(str) + " months"
# keep MORTGAGE / RENT / OWN; group the tiny remaining categories as OTHER
d["home"] = d["home_ownership"].where(d["home_ownership"].isin(["MORTGAGE", "RENT", "OWN"]), "OTHER")
d["interest rate"] = quintile(d["int_rate_pct"])      # interest-rate fifth (Q1 = lowest rates)
d["loan-to-income"] = quintile(d["loan_to_income"])  # loan-to-income fifth
d["annual income"] = quintile(d["annual_inc"])       # income fifth
d["balance-to-income"] = quintile(d["balance_to_income"]) # balance-to-income fifth
d["loan amount"] = quintile(d["loan_amount"])        # loan-amount fifth

CHARACTERISTICS = {                                # characteristics to analyse: display label -> column in d
    "grade": "grade", "term": "term", "interest rate": "interest rate", "loan-to-income": "loan-to-income",
    "annual income": "annual income", "balance-to-income": "balance-to-income", "loan amount": "loan amount",
    "home ownership": "home", "application type": "application_type",
}

resolved = d[d["status_group"] != "active"]          # loans with a known outcome: paid off or defaulted
active = d[d["status_group"] == "active"]            # open loans: these can still be late
overall_default = 100 * resolved["is_default"].mean() # overall default rate (%) on resolved loans
overall_late = 100 * active["is_late"].mean()         # overall late rate (%) on active loans

parts = []                                           # one summary table per characteristic, combined at the end
for label, col in CHARACTERISTICS.items():          # repeat for each characteristic
    # per bucket: number of resolved loans and their default rate (%)
    r = resolved.groupby(col, observed=True)["is_default"].agg(resolved_loans="size", default_rate_pct=lambda s: 100 * s.mean())
    # per bucket: number of active loans and their late rate (%)
    a = active.groupby(col, observed=True)["is_late"].agg(active_loans="size", late_rate_pct=lambda s: 100 * s.mean())
    # put the two side by side; the bucket becomes a normal column named 'bucket'
    t = r.join(a, how="outer").reset_index().rename(columns={col: "bucket"})
    t.insert(0, "characteristic", label)             # add a first column naming the characteristic
    parts.append(t)                                 # store this characteristic's table
drivers = pd.concat(parts, ignore_index=True)        # stack all the tables into one
drivers["bucket"] = drivers["bucket"].astype(str)    # bucket labels as plain text
```

```

# default rate relative to the overall rate (2.0 = twice the average)
drivers["default_lift"] = drivers["default_rate_pct"] / overall_default
drivers = drivers.round(2) # round to 2 decimals

# ---- ranking: how far apart are the best and worst bucket of each characteristic? ----
# Buckets with fewer than MIN_N loans are ignored here (a 62-loan bucket can show any rate by chance).
MIN_N = 500 # smallest bucket size we trust
# flag buckets that are too small to trust (true/false)
drivers["small_bucket"] = (drivers["resolved_loans"] < MIN_N) | (drivers["active_loans"] < MIN_N)

# default rates of the large-enough buckets, grouped by characteristic
big_r = drivers[drivers["resolved_loans"] >= MIN_N].groupby("characteristic")["default_rate_pct"]
# late rates of the large-enough buckets, grouped by characteristic
big_a = drivers[drivers["active_loans"] >= MIN_N].groupby("characteristic")["late_rate_pct"]
ranking = pd.DataFrame({ # one row per characteristic
    # lowest and highest default rate across a characteristic's buckets
    "default_lowest": big_r.min(), "default_highest": big_r.max(),
    "late_lowest": big_a.min(), "late_highest": big_a.max(), # lowest and highest late rate
})
# spread = highest minus lowest bucket rate, in percentage points: how strongly it separates risk
ranking["default_spread_pts"] = ranking["default_highest"] - ranking["default_lowest"]
# the same spread for the late rate
ranking["late_spread_pts"] = ranking["late_highest"] - ranking["late_lowest"]
# strongest separators first; round to 1 decimal
ranking = ranking.sort_values("default_spread_pts", ascending=False).round(1)

# print the overall rates for reference
print(f"overall default rate (resolved): {overall_default:.2f}% | overall late rate (active): {overall_late:.2f}%")
ranking

```

What this calculates

For each of nine characteristics (grade, loan term, interest rate, loan-to-income, annual income, balance-to-income, loan amount, home ownership, application type), the default rate among resolved loans and the late rate among active loans inside each bucket, and a ranking of which characteristic separates risky from safe loans best.

How it works, step by step

- **Sorting by loan_id first** makes the result identical on every run. Many loans share the same rate or income, and the bucket-cutting step splits such ties in row order; sorting first fixes the order.
- **quintile():** `s.rank(method="first")` numbers the loans 1, 2, 3, ...; `pd.qcut(..., 5)` then cuts them into five equal-sized groups labelled Q1 (lowest) to Q5 (highest). Categories such as grade and term are used as they are; home ownership values other than MORTGAGE, RENT and OWN are grouped as OTHER.
- **The loop** takes each characteristic in turn. For resolved loans it counts the loans in each bucket and computes 100 x mean of `is_default`, the default rate; for active loans it does the same with `is_late`, the late rate. The two results are joined side by side, labelled with the characteristic name, and stacked into one table, `drivers`.
- **default_lift** = the bucket's default rate / the overall default rate (18.96%). A lift of 2.0 means twice as risky as average. Example: grade G, 48.14 / 18.96 = 2.54.
- **The ranking:** for each characteristic, take the lowest and highest bucket rate and subtract: the **spread** in percentage points. A big spread means the characteristic separates risk well. Buckets with fewer than 500 loans are ignored in the ranking (`MIN_N = 500`), because tiny groups can show any rate by chance; for example the 62 resolved 'Direct Pay' loans had a 46.8% default rate.

What the result shows

The ranking table below, then the full table of all 41 buckets. The overall default rate on resolved loans is 18.96% and the overall late rate on active loans is 3.19%.

The idea to deliver

A first, easily understood ranking of the characteristics by how strongly they separate good loans from bad.

characteristic	default lowest	default highest	late lowest	late highest	default spread pts	late spread pts
grade	5.8	48.1	1.0	14.6	42.3	13.6
interest rate	6.0	33.8	1.0	6.3	27.8	5.4
loan-to-income	12.0	28.5	2.7	3.9	16.5	1.2

characteristic	default lowest	default highest	late lowest	late highest	default spread pts	late spread pts
term	15.0	31.4	3.0	3.6	16.4	0.7
annual income	14.5	22.6	3.0	3.5	8.1	0.5
loan amount	14.2	22.3	2.7	3.6	8.1	0.9
balance-to-income	15.9	23.1	2.7	3.6	7.3	0.8
home ownership	16.3	22.3	2.9	3.5	6.0	0.6
application type	17.8	19.0	3.2	3.2	1.1	0.0

- **Grade** separates most: default 5.8% for grade A up to 48.1% for grade G (spread 42.3 points); late rate 1.0% to 14.6%.
- **Interest rate** is next: 6.0% default in the lowest fifth up to 33.8% in the highest fifth (27.8 points). Grade and rate carry almost the same information because the lender sets the rate from the grade.
- **Loan-to-income** (12.0% to 28.5%) and **term** (15.0% for 36 months, 31.4% for 60 months) are next, then income (22.6% in the lowest fifth against 14.5% in the highest).
- **Application type** hardly matters (17.8% joint against 19.0% individual). Late rates barely move for anything except grade and rate.

The full table of all buckets follows (default and late rates for every bucket):

#	characteristic	bucket	resolved loans	default rate pct	active loans	late rate pct	default lift	small bucket
0	grade	A	16671	5.81	41200	0.98	0.31	False
1	grade	B	26897	12.27	52175	2.29	0.65	False
2	grade	C	26580	20.94	49260	3.70	1.10	False
3	grade	D	14494	28.48	24382	5.51	1.50	False
4	grade	E	6666	38.12	6763	8.34	2.01	False
5	grade	F	2245	46.15	1760	11.76	2.43	False
6	grade	G	671	48.14	535	14.58	2.54	False
7	term	36 months	71590	15.02	118182	2.97	0.79	False
8	term	60 months	22634	31.43	57893	3.63	1.66	False
9	interest rate	Q1 (lowest)	16836	6.02	37224	0.96	0.32	False
10	interest rate	Q2	16930	10.96	37130	2.03	0.58	False
11	interest rate	Q3	19866	17.47	34193	2.98	0.92	False
12	interest rate	Q4	20007	22.83	34053	4.00	1.20	False
13	interest rate	Q5 (highest)	20585	33.79	33475	6.34	1.78	False
14	loan-to-income	Q1 (lowest)	19117	12.02	34943	2.68	0.63	False
15	loan-to-income	Q2	19164	15.19	34896	2.76	0.80	False
16	loan-to-income	Q3	19540	17.54	34519	3.16	0.93	False
17	loan-to-income	Q4	19588	22.61	34472	3.41	1.19	False
18	loan-to-income	Q5 (highest)	16815	28.53	37245	3.89	1.50	False
19	annual income	Q1 (lowest)	19215	22.58	34845	3.46	1.19	False
20	annual income	Q2	19813	20.76	34247	3.22	1.10	False
21	annual income	Q3	19374	19.25	34685	3.19	1.02	False
22	annual income	Q4	18665	17.14	35395	3.09	0.90	False
23	annual income	Q5 (highest)	17157	14.47	36903	3.00	0.76	False
24	balance-to-income	Q1 (lowest)	18110	18.50	35950	3.24	0.98	False
25	balance-to-income	Q2	18687	23.12	35373	3.57	1.22	False
26	balance-to-income	Q3	18972	20.86	35087	3.24	1.10	False
27	balance-to-income	Q4	19537	16.55	34523	2.72	0.87	False
28	balance-to-income	Q5 (highest)	18918	15.86	35142	3.17	0.84	False

#	characteristic	bucket	resolved loans	default rate pct	active loans	late rate pct	default lift	small bucket
29	loan amount	Q1 (lowest)	19946	14.18	34114	2.69	0.75	False
30	loan amount	Q2	19840	17.56	34220	3.05	0.93	False
31	loan amount	Q3	19862	19.98	34197	3.07	1.05	False
32	loan amount	Q4	19272	21.67	34788	3.44	1.14	False
33	loan amount	Q5 (highest)	15304	22.26	38756	3.63	1.17	False
34	home ownership	MORTGAGE	47151	16.28	86203	2.88	0.86	False
35	home ownership	OTHER	56	7.14	299	3.34	0.38	True
36	home ownership	OWN	10243	19.45	20684	3.35	1.03	False
37	home ownership	RENT	36774	22.28	68889	3.52	1.18	False
38	application type	Direct Pay	62	46.77	241	8.71	2.47	True
39	application type	Individual	92346	18.96	158856	3.18	1.00	False
40	application type	Joint	1816	17.84	16978	3.20	0.94	False

10.3 Lens 2: a predictive model that weighs everything together

Looking at one characteristic at a time can mislead, because characteristics overlap: high-rate loans are also more often 60-month loans, so is it the rate or the term that matters? A **logistic regression** is a standard statistical model that predicts a yes/no outcome from several characteristics at once and shows how much each one matters *while the others are held fixed*. Two models were built: one predicting default (on the 94,224 resolved loans) and one predicting late payment (on the 176,075 active loans). Grade was left out because the interest rate is derived from it.

Hex Python cell

```
# Task 5, step 2: which characteristics predict default and late payment, taken together?
# Two logistic-regression models, each scored on a held-out 30% test set:
#   default model: resolved loans only (paid off vs defaulted); late model: active loans only (current vs late).
# Grade is left out of the models because the interest rate is derived from it; its stand-alone strength is still shown.
# Needs task4_customers (SQL cell) and X (Task 4 preparation cell). Outputs: model_summary, default_table, late_table.

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
    » 1 = perfect)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# numerical functions (exp, maximum, where)
# tables (DataFrames)
# the model: predicts a yes/no outcome from several features at once
# AUC: how well scores rank risky loans above safe ones (0.5 = coin flip,

base = task4_customers.copy()
base["is_default"] = base["is_default"].astype(int)
base["is_late"] = base["is_late"].astype(int)

# splits the data into a training part and a held-out test part
# rescales each feature to mean 0, std 1

feat = X.copy()
    » preparation cell)
feat["term_60"] = (base["term_months"] == 60).astype(float)
feat = feat.drop(columns=["term_months"])
# 1.0 if the borrower rents (MORTGAGE is the baseline)
feat["home_RENT"] = (base["home_ownership"] == "RENT").astype(float)
# 1.0 if the borrower owns outright
feat["home_OWEN"] = (base["home_ownership"] == "OWN").astype(float)
# 1.0 for a joint application
feat["joint_application"] = (base["application_type"] == "Joint").astype(float)
# grade as a number, A=1 ... G=7 (only for the stand-alone comparison; not a model input)
grade_num = base["grade"].map({g: i for i, g in enumerate("ABCDEFG", start=1)})

# start from X: capped and log-transformed, not yet standardised (Task 4)
# 1.0 for a 60-month loan, 0.0 for a 36-month loan
# remove the raw term column (replaced by term_60)

def fit_and_report(mask, target):
    Xm, y = feat[mask], base.loc[mask, target]
    # 70% training / 30% held-out test; stratify keeps the same event share in both; fixed seed
    X_tr, X_te, y_tr, y_te = train_test_split(Xm, y, test_size=0.3, random_state=42, stratify=y)
    scaler = StandardScaler().fit(X_tr)
    # fit on the scaled training data (max_iter=1000 = enough steps to converge)
    model = LogisticRegression(max_iter=1000).fit(scaler.transform(X_tr), y_tr)
    # AUC on the held-out test data: predicted probability of the event vs what actually happened
    test_auc = roc_auc_score(y_te, model.predict_proba(scaler.transform(X_te))[:, 1])

    # AUC of each feature used ALONE as a risk score (0.5 = no signal)
    alone = {c: roc_auc_score(y_te, X_te[c]) for c in X_te.columns}
    # the same for grade, on the same test loans
    alone["grade (A=1 ... G=7)"] = roc_auc_score(y_te, grade_num.loc[X_te.index])
    table = pd.DataFrame({"auc_alone": pd.Series(alone)})
    # odds ratio: how much a 1-standard-deviation rise multiplies the odds of the event, other features held fixed (above 1 = riskier)
    table["odds_ratio_per_1sd"] = pd.Series(np.exp(model.coef_[0]), index=X_te.columns)
```

```
# strength ignoring direction: an AUC of 0.45 is as informative as 0.55, just reversed
table["auc_alone_strength"] = np.maximum(table["auc_alone"], 1 - table["auc_alone"])
# direction: does a higher value of the feature go with more risk?
table["higher_value_means"] = np.where(table["auc_alone"] >= 0.5, "riskier", "safer")
# strongest features first; round to 3 decimals
table = table.sort_values("auc_alone_strength", ascending=False).round(3)
# one-row summary: target, loans modelled, event rate (%), test rows, and the model's test AUC
summary = {"target": target, "loans_modelled": len(Xm), "event_rate_pct": round(100 * y.mean(), 2),
           "test_rows": len(y_te), "model_test_auc": round(test_auc, 3)}
return summary, table # hand both results back to the caller

# default model: resolved loans only, predicting default
s1, default_table = fit_and_report(base["status_group"] != "active", "is_default")
# late model: active loans only, predicting late payment
s2, late_table = fit_and_report(base["status_group"] == "active", "is_late")
model_summary = pd.DataFrame([s1, s2]) # combine the two summaries into one table
model_summary
```

What this calculates

For each of the two outcomes: how well a model using nine characteristics predicts it on loans it has never seen (the AUC score), how strong each characteristic is when used alone, and its odds ratio inside the model.

How it works, step by step

- **base["is_default"].astype(int)** turns true/false into 1/0, the target the model learns to predict.
- **feat** starts from **X** (the capped and log-transformed features of Task 4) and adds three yes/no columns: **term_60** (1 if the loan is 60 months), **home_RENT** and **home_OWN** (1 if the borrower rents or owns; mortgage is the reference), and **joint_application**.
- **train_test_split(..., test_size=0.3, stratify=y, random_state=42)**: 70% of the loans are used to train the model and 30% are held back to test it. **stratify** keeps the same share of defaults in both parts; **random_state** fixes the split.
- **StandardScaler().fit(X_tr)** learns each feature's mean and spread from the training part only, and then rescales both parts. (Learning them from all data would leak information about the test loans into training.)
- **LogisticRegression(max_iter=1000)** fits the model, allowing up to 1,000 steps to converge.
- **AUC (roc_auc_score)**: take one loan that had the event and one that did not; the AUC is the chance the model gives the event loan the higher risk score. 0.5 is a coin flip and 1.0 is perfect. **predict_proba(...)[:, 1]** gives each loan's predicted probability of the event.
- **auc_alone** scores each characteristic on its own as a risk score. A value below 0.5 (for example 0.446 for income) means higher values go with *lower* risk, so the code also reports the strength ignoring direction, **max(auc, 1 - auc)**, and the direction.
- **odds_ratio_per_1sd** = **exp(model coefficient)**. The odds of an event are its chance divided by the chance it does not happen. A value of 1.63 means that a one-standard-deviation increase in the characteristic multiplies the odds by 1.63, with the others held fixed; below 1 means safer. For yes/no columns, one standard deviation is not a clean yes-versus-no jump, so read them as direction and rough size.

What the result shows

The summary table below, then one table per outcome.

The idea to deliver

A test of whether the ranking from Lens 1 survives when characteristics compete with each other, and an honest measure of how good the prediction really is.

row	target	loans modelled	event rate pct	test rows	model test auc
0	is_default	94224	18.96	28268	0.697
1	is_late	176075	3.19	52823	0.674

Default model: single-characteristic strength and odds ratios

characteristic	auc alone	odds ratio per 1sd	auc alone strength	higher value means
grade (A=1 ... G=7)	0.682	NaN	0.682	riskier
int_rate_pct	0.681	1.634	0.681	riskier
loan_to_income	0.604	1.230	0.604	riskier
term_60	0.594	1.244	0.594	riskier
annual_inc	0.446	0.965	0.554	safer
loan_amount	0.551	0.939	0.551	riskier
home_RENT	0.545	1.135	0.545	riskier
balance_to_income	0.465	0.935	0.535	safer
joint_application	0.498	0.944	0.502	safer
home_OWEN	0.500	1.036	0.500	riskier

Late-payment model: single-characteristic strength and odds ratios

characteristic	auc alone	odds ratio per 1sd	auc alone strength	higher value means
grade (A=1 ... G=7)	0.672	NaN	0.672	riskier
int_rate_pct	0.668	1.756	0.668	riskier
loan_to_income	0.548	1.069	0.548	riskier
loan_amount	0.538	1.056	0.538	riskier
term_60	0.529	0.874	0.529	riskier
home_RENT	0.519	1.058	0.519	riskier
annual_inc	0.491	1.038	0.509	safer
balance_to_income	0.491	0.999	0.509	safer
joint_application	0.503	0.956	0.503	riskier
home_OWEN	0.500	1.039	0.500	safer

Column guide: **auc alone** = how well the characteristic ranks risky above safe loans when used by itself (0.5 = no signal); **odds ratio per 1sd** = its odds ratio inside the model (NaN for grade, because grade is not a model input); **auc alone strength** = the same score ignoring direction; **higher value means** = whether a higher value goes with more risk.

- **The full model reaches an AUC of 0.697 for default and 0.674 for late payment:** moderate, not strong. 0.5 would be a coin flip.
- **Grade alone scores 0.682 and interest rate alone 0.681**, almost as much as everything together. All the other characteristics add only about 0.02.
- **Odds ratios for default:** interest rate 1.63, term (60 months) 1.24, loan-to-income 1.23, renting 1.14. Loan amount looked slightly riskier on its own (AUC 0.551) but its odds ratio is 0.94 once the others are held fixed, so its apparent effect came from overlap with rate and term. Balance-to-income (0.94) and income (0.97) are weakly on the safer side.
- **Late payment** is predicted almost only by interest rate (odds ratio 1.76); the other odds ratios lie between about 0.87 and 1.07.

What to say about the model

The lender's own grade and interest rate already contain most of what this data can tell us about risk: existing underwriting captures most of the accessible signal. An AUC of about 0.70 also means we should not present this as a precise risk model. The interest rate reflects the lender's own view of the borrower, so it predicts default without necessarily causing it.

10.4 Lens 3: how would simple lending rules have performed?

A rule here means a simple flag a lender could apply at the time of application, such as "grade E, F or G". Every rule below uses only information known at application time (grade, interest rate, term, loan-to-income), so none of them peeks at the outcome. Each rule is judged on history by four questions: how many loans it flags, how risky the flagged loans are compared with the rest, what share of all defaults it catches, and what share of good loans it wrongly flags.

Hex Python cell

```
# Task 5, step 3: how would candidate lending rules have performed on historical loans?
# Each rule flags loans using only information known at application time (grade, rate, term, loan-to-income).
# For each rule: default rate inside vs outside it, share of all defaults it catches, share of good loans it would
# also flag (its cost), and the late rate on active loans inside vs outside. Output: rules_table (one row per rule).

import pandas as pd                                     # tables (DataFrames)

d = task4_customers.copy()                             # SQL result from Hex; a copy keeps the original unchanged
d["is_default"] = d["is_default"].astype(bool)          # make sure the flag is true/false (needed for ~ and sums below)
d["is_late"] = d["is_late"].astype(bool)               # same for the late flag

rules = {                                                # each rule is a true/false test per loan: True = the loan is flagged
    "Grade D-G": d["grade"].isin(list("DEFG")),        # grade D, E, F or G
    "Grade E-G": d["grade"].isin(list("EFG")),         # grade E, F or G
    "Rate >= 20%": d["int_rate_pct"] >= 20,            # interest rate of 20% or more
    "Rate >= 25%": d["int_rate_pct"] >= 25,            # interest rate of 25% or more
    "# 60-month term AND rate of 16% or more (& = both conditions must hold)",
    "60-month AND rate >= 16%": (d["term_months"] == 60) & (d["int_rate_pct"] >= 16),
    "Loan-to-income >= 0.35": d["loan_to_income"] >= 0.35, # loan amount is at least 35% of annual income
    "# 60-month term AND loan-to-income of 0.30 or more",
    "60-month AND loan-to-income >= 0.30": (d["term_months"] == 60) & (d["loan_to_income"] >= 0.30),
    "# either condition is enough (| = or): grade D-G, or 60-month with loan-to-income of 0.30 or more",
    "Grade D-G OR (60-month AND LTI >= 0.30)": d["grade"].isin(list("DEFG")) | ((d["term_months"] == 60) & (d["loan_to_income"] >= 0.30)),
}

resolved = d["status_group"] != "active"               # loans with a known outcome: paid off or defaulted
active = ~resolved                                     # open loans: these can still be late (~ = not)
total_defaults = d.loc[resolved, "is_default"].sum()    # all defaults among resolved loans (True counts as 1)
total_good = (~d.loc[resolved, "is_default"]).sum()    # all paid-off loans (resolved loans that did not default)

rows = []
for name, flag in rules.items():
    fr, nr = d[resolved & flag], d[resolved & ~flag]    # will collect one result row per rule
    fa, na = d[active & flag], d[active & ~flag]        # for each rule: its name and its true/false test
    rows.append({                                       # resolved loans flagged (fr) and not flagged (nr)
        "rule": name,                                  # active loans flagged (fa) and not flagged (na)
        "# % of resolved loans the rule flags",        # one row of results for this rule
        "flagged_pct_of_resolved": 100 * len(fr) / resolved.sum(),
        "# default rate (%) among flagged resolved loans",
        "default_rate_flagged": 100 * fr["is_default"].mean(),
        "# default rate (%) among resolved loans NOT flagged",
        "default_rate_not_flagged": 100 * nr["is_default"].mean(),
        "# % of ALL defaults that the rule flags",
        "defaults_caught_pct": 100 * fr["is_default"].sum() / total_defaults,
        "# % of paid-off loans the rule would also flag (the cost of the rule)",
        "good_loans_flagged_pct": 100 * (~fr["is_default"]).sum() / total_good,
        "late_rate_flagged": 100 * fa["is_late"].mean(), # late rate (%) among flagged active loans
        "late_rate_not_flagged": 100 * na["is_late"].mean(), # late rate (%) among active loans NOT flagged
    })

rules_table = pd.DataFrame(rows).round({                # turn the rows into a table; round each column
    "# decimals per column: 1 for most, 2 for the late rates
    "flagged_pct_of_resolved": 1, "default_rate_flagged": 1, "default_rate_not_flagged": 1,
    "defaults_caught_pct": 1, "good_loans_flagged_pct": 1, "late_rate_flagged": 2, "late_rate_not_flagged": 2,
})

# overall rates for reference
print(f"overall default rate (resolved): {100 * d.loc[resolved, 'is_default'].mean():.1f}% | "
      f"overall late rate (active): {100 * d.loc[active, 'is_late'].mean():.2f}%")
rules_table                                           # last line of the cell: Hex displays the table
```

What this calculates

For eight candidate rules: the share of loans flagged, the default rate inside and outside the rule, the share of all defaults caught, the share of paid-off loans that would also be flagged, and the late rate inside and outside.

How it works, step by step

- rules** is a dictionary of named true/false tests. `d["grade"].isin(list("DEFG"))` is true for loans with grade D, E, F or G. The symbols `&`, `|` and `~` mean "and", "or" and "not" and are applied to whole columns at once; the brackets are required because these symbols bind more tightly than comparisons.
- resolved** and **active** split the loans into those with a known outcome (used for default) and open ones (used for lateness). **total_defaults** counts all defaults; **total_good** counts all paid-off loans.

- **For each rule:** fr / nr are the resolved loans flagged and not flagged; fa / na are the active loans flagged and not flagged.
- **flagged_pct_of_resolved** = $100 \times \text{flagged resolved loans} / \text{all resolved loans}$.
- **default_rate_flagged** and **default_rate_not_flagged** = $100 \times \text{average of is_default inside and outside the rule}$. A good rule has a much higher rate inside.
- **defaults_caught_pct** = $100 \times \text{defaults among flagged loans} / \text{all defaults}$ (in statistics this is called recall or sensitivity).
- **good_loans_flagged_pct** = $100 \times \text{paid-off loans among flagged loans} / \text{all paid-off loans}$. This is the **cost** of the rule: good customers who would be flagged too (the false-alarm rate).
- **late_rate_flagged** / **late_rate_not_flagged** = the same comparison for lateness on active loans.

What the result shows

The rules table below. This cell prints exactly this table; the numbers below were computed with the same code on the full dataset. (If this cell is not yet in your Hex notebook, add it and check that its output matches the table.)

The idea to deliver

It turns statistics into decisions a lender could take, and shows the trade-off honestly: catching more defaults costs more false alarms.

Rule	Flagged (% of resolved)	Default: flagged	Default: not flagged	Defaults caught	Good loans flagged	Late: flagged	Late: not flagged
Grade D-G	25.6	33.3	14.0	44.9	21.0	6.56	2.40
Grade E-G	10.2	40.7	16.5	21.8	7.4	9.37	2.85
Rate 20% or more	8.5	37.0	17.3	16.5	6.6	7.47	2.75
Rate 25% or more	2.1	40.4	18.5	4.5	1.6	9.17	3.03
60-month AND rate 16% or more	12.4	39.0	16.1	25.5	9.3	5.89	2.81
Loan-to-income 0.35 or more	13.9	29.2	17.3	21.4	12.2	3.85	3.05
60-month AND loan-to-income 0.30 or more	10.1	37.1	16.9	19.8	7.8	3.95	3.07
Grade D-G OR (60-month AND LTI 0.30 or more)	29.6	32.5	13.3	50.8	24.7	5.24	2.40

All figures are percentages; overall the default rate on resolved loans is 18.96% and the late rate on active loans is 3.19%. LTI = loan-to-income.

- **Every rule separates risk, but none is clean.** Even the best rules default only about 40% of the time, so roughly 60% of the flagged loans are fine. That is why they should trigger a review, tighter pricing or lower limits, not an automatic rejection.
- **Efficient rules:** "Grade E-G" and "60-month AND rate 16% or more" flag 10-12% of loans that default about 39-41% of the time, against roughly 16% for the rest.
- **The wide rule** "Grade D-G" catches 44.9% of all defaults but also flags 21.0% of good loans: a bigger net catches more but costs more.
- **Loan-to-income** separates default (29.2% against 17.3%) but hardly separates lateness (3.85% against 3.05%).

10.5 The written answers to the two questions in the brief

Question 1: What customer or loan characteristics best predict default or late payment?

- **Grade and interest rate, by far.** Default rises from 5.8% (grade A) to 48.1% (grade G); by rate fifth from 6.0% to 33.8%. Late rate rises from 1.0% (A) to 14.6% (G). Alone each scores an AUC of about 0.68 (default) and 0.67 (late); everything together reaches 0.697 and 0.674.

- **Loan term:** 60-month loans default at 31.4% against 15.0% for 36-month loans, and this remains when other factors are held fixed (odds ratio 1.24).
- **Loan-to-income:** default rises from 12.0% (lowest fifth) to 28.5% (highest fifth), odds ratio 1.23; it does not help predict lateness.
- **Weaker signals:** lower income (22.6% default in the lowest fifth against 14.5% in the highest) and renting (22.3% against 16.3% for mortgage holders).
- **No useful signal:** joint against individual application (17.8% against 19.0%), and, once other factors are considered, loan amount and balance-to-income.
- **Late payment is harder to predict than default:** only interest rate and grade separate late loans meaningfully.

Question 2: If you were designing lending rules, what would you flag or change?

- **Flag grade E-G and 60-month loans priced at 16% or more** for tighter limits, higher pricing or manual review: about 10-12% of loans that default at about 39-41%.
- **Treat high loan-to-income (0.35 or more) as a secondary default flag**, for example by capping the loan size or requiring income verification. It adds little for lateness.
- **Add early-warning monitoring for high-rate loans:** loans at 20% or more are late at 7.47% against 2.75%.
- **Do not add rules for joint applications or loan amount alone:** no useful signal.
- **Use rules as triggers, not automatic declines:** even the best rules are wrong for about 60-67% of the loans they flag.
- **Fix the data, the biggest change:** record payment dates and days late, credit limits and utilisation, outcome dates and rejected applications, so that lateness and default can be measured directly and rules tested properly.

Limitations to state

Default rates use resolved loans only and the statuses appear not to share one snapshot date, so rates are for comparing groups, not exact risk levels.

No payment history: 'late' means late at the time of the snapshot.

Only approved loans are in the data; we cannot know how rejected applicants would have behaved.

The rule thresholds (16%, 0.30, 0.35) were chosen by eye from the buckets and tested on the same data; they need validating on new data.

These are associations, not causes: the interest rate reflects the lender's own risk view.

11. Limitations, adaptations and quality control

11.1 The adaptations, in one table

Asked for	Not possible because	What we did	How good a substitute is it?
Join loans and payments (Task 3)	No payments table, no payment dates	Late = current status is grace period, 16-30 or 31-120 days late	Good for 'who is late now'; cannot show payment history or days late.
Late-payment frequency per customer (Task 3)	One loan per customer	Share of active loans that are late within a rate band and term	Answers the same question at group level.
Average delay (Task 4)	No days-late data	Not used; loan amount, rate, term, income and ratios used instead	Groups describe borrowers, not payment behaviour.
Credit utilisation (Task 4)	No credit limits	Balance-to-income as a leverage proxy	Rough: balances include mortgages, so it is not utilisation.
A clean default rate by year (Task 2)	Recent loans are still open; statuses seem to have different snapshot dates	Two default rates: against all loans and against resolved loans only	Resolved-only is fairer for 2012-2017; 2018-2019 remain unreliable.

11.2 How correctness was protected

- **Independent verification.** Before any number was written down, it was recomputed in Python directly from the CSV files. The BigQuery and Hex results were then compared cell by cell.
- **Load validation.** Row counts, sums, blank counts and join integrity were checked after loading (Section 4.4), and again for the view (Section 4.6).
- **One source of truth.** Definitions live only in the view, so tasks cannot disagree with each other. For example, Task 1's year totals equal Task 2's loan counts.
- **Tie-outs.** Group counts add up to 270,299 (outcome groups), 176,075 (active loans), 5,614 (late loans) and 94,224 (resolved loans) wherever they appear.
- **Reproducibility.** Fixed random seeds, sorting before bucketing, and code stored as files in the public repository.
- **Honesty about weaknesses.** Every caveat (snapshot dates, approved loans only, workarounds) is written in the notebook itself.

11.3 What would be done next with more time or data

- Obtain **payment history** (dates and amounts) and **credit limits**: this would allow real lateness measures, true credit utilisation and a much better model.
- Obtain **rejected applications** so that the models are not limited to approved loans.
- Understand and fix the **snapshot dates** so that year-to-year comparison is clean; a survival analysis (a method that handles loans that have not finished yet) would then be the right tool for default timing.
- Test the rule thresholds on **new data** (loans from a later period) and estimate the money effect (lost interest against avoided losses).
- Try a **more flexible model** (for example gradient boosting) to see whether the modest AUC of 0.70 can be improved, while keeping the simple logistic model as the explainable benchmark.
- Have any rule reviewed for **fairness and regulation** before use: it must not depend directly or indirectly on protected characteristics.

12. Presentation script

This script is written to be **spoken**, in the first person, in simple language. Plan on about 14 minutes plus questions. Each part has a suggested time, a **Show** cue (what should be on screen) and a **Say** text. Adjust the wording to sound like you; keep the numbers. A ten-minute route is given at the end.

Part 1. Opening: what I was asked and what I found (about 1 minute)

Show: The top note of the Hex notebook.

"Thank you for your time. I was asked to analyse a public dataset of about 270,000 consumer loans, using SQL in BigQuery and Python in Hex, and to answer two business questions: what predicts a loan going bad, and what would I change in lending rules. Let me start with the answer, then show how I got there."

"In one sentence: the lender's own risk grade and interest rate are, by far, the strongest predictors of default and late payment; simple rules based on grade, term and loan size can flag risky loans, but every rule also flags good loans, so I would use them to trigger reviews and pricing, not automatic declines. I will also be open about the limits of the data, because they shaped the work."

Part 2. The data and the first surprise (about 1.5 minutes)

Show: The table in the top note, comparing what the brief assumed with what the data has.

"The dataset has six files. The core two are the loans and the customers, 270,299 rows each, and they match one-to-one, so each customer has exactly one loan. The other four are small helper files."

"The first surprise: there is no payments table. The brief asked me to join loans and payments to find late payers, but there are no payment dates and no days-late figure, and there are no credit limits either. So I adapted, and I documented every adaptation at the top of the notebook. For lateness I used the loan's current status: In Grace Period, 16 to 30 days late, or 31 to 120 days late. It is a snapshot, not a payment history, and I say so wherever it matters."

Part 3. What I did in BigQuery (about 1.5 minutes)

Show: Optional: BigQuery tables list and the v_loans query.

"I loaded each file unchanged into BigQuery with an explicit schema, and I validated the load: row counts, sums, blank counts and join checks, all compared with numbers I calculated independently from the files. Every check matched."

"Then I built one cleaned view, v_loans. It fixes the messy fields, for example a stray header row inside a lookup file, five spellings of application type, and a text code for missing values. It also holds the definitions: defaulted means Charged Off or Default, resolved means paid off or defaulted, and late means grace period or late. Every task reads that one view, so the definitions cannot drift."

Part 4. Task 1: outcomes by year (about 1.5 minutes)

Show: The two Task 1 charts.

"Loan volume grew about twenty-fold from 2,594 loans in 2012 to 51,737 in 2019. Older loans are mostly paid off or defaulted, and newer ones are mostly still active, which is expected."

"One thing stood out: 86% of the 2016 loans are still active, for both 36 and 60-month terms, although the latest loan was issued in December 2019 and a 36-month loan from 2016 should be finished. My working explanation is that the statuses were recorded at different dates for different years. That is a hypothesis, not a proven fact, but it means I must be careful when comparing years."

Part 5. Task 2: default rate, two ways (about 1.5 minutes)

Show: The Task 2 line chart.

"The average interest rate stayed between 12.6 and 14.5 percent and the average loan between about 13.7 and 16.5 thousand dollars, so neither explains the changes in default."

"The default rate depends on what you divide by. Against all loans, it drops from 18.2 percent for 2015 to 2.6 percent for 2016. That looks like a huge improvement, but it is misleading: open loans cannot have defaulted yet. Against resolved loans only, the rate barely moves, from 20.2 to 18.6 percent. The gap between the two lines is simply the share of unfinished loans. So the credit quality did not suddenly improve; the loans are younger."

Part 6. Task 3: late payments and interest rate (about 1.5 minutes)

Show: The Task 3 bubble chart.

"About 3.2 percent of active loans are late, 5,614 customers, and I listed them. The chart shows late rate against interest rate, split by 36 and 60 months, with bubble size showing how many loans stand behind each point."

"The message is clear: the higher the interest rate, the more likely a loan is late, from about 0.6 percent in the lowest band to about 15 percent in the highest, and the pattern holds within each term. The lender prices for risk, so this mostly shows that its pricing works. I do not claim high rates cause lateness. I also point out that the highest-rate bubbles are small, so single points there should not be over-read."

Part 7. Task 4: customer segments (about 1.5 minutes)

Show: The k comparison chart, then the segment bubble chart.

"Since delay and credit utilisation do not exist, I grouped customers by loan amount, interest rate, term, income, loan-to-income, and a leverage measure, total balances divided by income. I capped extreme values, took logarithms of the skewed features and standardised everything, without deleting anyone. I did not use default or late status to build the groups; I used them only to describe the groups afterwards."

"I tested two to eight groups. The quality score was about 0.2 for every choice from three to eight, meaning the customers form a continuum, not neat clumps. I chose five because it was the most stable across random restarts and gave five distinct, explainable types. The riskiest is a group of 60-month, higher-rate loans, with 32 percent default, then a low-income, stretched group at 23 percent. The safest groups are around 11 to 15 percent. Notably, the groups differ threefold in default rate even though default was not used to form them."

Part 8. Task 5: what predicts default, and the rules (about 2 minutes)

Show: The ranking table, the model tables and the rules table.

"I looked at it in three ways. First, default rate by characteristic: from grade A to grade G it climbs from 5.8 to 48.1 percent, and by interest-rate fifth from 6 to 34 percent. Term matters, 60 months against 36 months is 31 against 15 percent. Loan-to-income matters, 12 to 28.5 percent. Joint applications and loan amount hardly matter."

"Second, a statistical model that weighs everything together. It reaches an AUC of 0.70 for default and 0.67 for lateness, which is moderate, and grade alone already gets 0.68. In other words the lender's own grade and rate capture most of what this data can say."

"Third, I tested simple rules on history. Flagging grade E to G covers 10 percent of loans that default 41 percent of the time against 16.5 for the rest; flagging 60-month loans with a rate of 16 percent or more is similar. But even these rules are wrong about six times in ten for the loans they flag, so I recommend them as triggers for review, tighter pricing or lower limits, plus early-warning monitoring for loans above 20 percent, not automatic declines."

Part 9. Challenges and limitations (about 1.5 minutes)

Show: The top note again.

"The biggest challenge was that the data did not match the brief, so I documented each workaround. The second was the snapshot inconsistency, which I handled by showing two default rates and labelling unreliable years. Along the way I also fixed an order-dependence bug that made numbers shift by a tenth of a point between runs, and I validated everything against independent calculations."

"The limits: no payment history, only approved loans, resolved-only default rates, and rule thresholds tested on the same data. These are associations, not causes."

Part 10. Closing (about 1 minute)

Show: The recommendations list.

"To summarise: grade and interest rate carry most of the signal; term and loan-to-income add some; the recommended rules are simple and explainable, but they must be used as triggers. The most valuable next step is better data: payment history, credit limits, outcome dates and rejected applications. Everything is in a public repository, so it can be reproduced. I am happy to take questions."

If you only have ten minutes

Keep Parts 1, 2, 5, 6, 8 and 10 in full; shorten Part 3 to two sentences; skip Part 4's detail but keep the 2016 remark; give Part 7 only the sentence about five segments differing threefold in default without using default to form them; shorten Part 9 to the two biggest challenges.

Delivery tips

Say each headline number once, clearly, and let the chart carry the rest. Always name the caveat right after a strong-sounding number (for example, after the 2.6 percent, say why it is misleading).

If you do not know something, say so plainly and say how you would find out. The reviewers are data-literate and will respect honesty about the 2016 puzzle more than a forced explanation.

Refer to the top note of the notebook whenever you are asked why something differs from the brief.

13. Questions the reviewers may ask, with answers

The questions are grouped by theme. Each answer is written so that you can say it almost as it stands. Where an answer contains a number, it comes from the verified results in this guide.

13.1 The data and the approach

Q1. Why did you not do Task 3 the way the brief describes, joining loans and payments?

Because there is no payments table in the data: only customer, loan and four small helper files. There are also no payment dates or days-late figures, so a join to payments was impossible. I confirmed this by profiling every file before I started, and I told the reviewers up front in the notebook.

Instead I used the loan's current status as the signal of lateness (grace period, 16-30 days late, 31-120 days late) and measured how the late share changes with interest rate, which answers the same business question at group level. If a payments table existed, I would compute actual days late per customer and redo Tasks 3 and 4 with it.

Q2. How did you decide what "late" means? Isn't a status snapshot a weak definition?

Yes, it has limits, and I state them. It is a picture of where each loan stands at one moment: a borrower who was late last year and has since caught up is not counted, and there is no measure of how many payments were missed or how many days late.

I chose the three statuses that explicitly describe overdue or borderline payment. I excluded Charged Off and Default because those are more severe and are already counted as defaulted. I also measured the late rate against active loans only, since a paid-off or defaulted loan can no longer become late.

Q3. Why did you load the raw data unchanged and clean it later in a view?

It is the standard practice in data warehousing. The raw tables stay identical to the source, so anyone can verify them and go back to the original. All cleaning rules then sit in one visible place, the view `v_loans`, so they are easy to audit and to change. If I had cleaned before loading, the rules would be scattered across files and could not easily be checked.

Q4. How do you know the data in BigQuery matches the source files?

I wrote validation queries whose expected answers I calculated independently in Python from the original CSV files: row counts for all six tables, sums of loan amount, funded amount, installment, income and balances, counts of blank values, the number of distinct customers, and a two-way join check between loans and customers. Every check matched, and I repeated the process for the view, with 17 checks plus a cross-check of the region against the separate region file.

Q5. What data quality problems did you find?

Nine main ones: customer identifiers stored as unreadable hashed text; the text `n/a` used as a missing-value code (18,745 rows); a stray header row inside the state lookup; five spellings of application type; a leading space in the loan term; duplicate or constant columns (`issue_date`, `notes`, `funded_amount`); extreme values (incomes from \$34 to \$9.55 million, loan-to-income up to 175); joint income blank for 93% of customers (by design); and files that changed while I was reading them. I handled each in the view or by capping, and I deleted no customers.

Q6. Why does 2016 look so strange, and what did you do about it?

86.1% of 2016 loans are still active, for both 36-month and 60-month loans, although the latest loan was issued in December 2019, and a 36-month loan from 2016 should have finished by then. 2014 is odd too: 29% of its 36-month loans are still active, against 0% for 2013 and 2015.

My working hypothesis, which I have not proven, is that the statuses were recorded at different dates for different years. What I did about it: I compared default rates in two ways (against all loans and against resolved loans only), labelled the 2018-2019 resolved-only rates as unreliable, and wrote the caveat in the notebook. If asked for the cause, I would say I cannot confirm it from this data and would ask the data owner for the snapshot date of each cohort.

Q7. Is this data representative of the real market? Where does it come from?

The column names and formats resemble the public LendingClub loan data, an American online lender, but the files include no data dictionary and no statement of origin, so that is only a hypothesis. The data contains only loans that were approved and issued, so it does not represent everyone who applied. My conclusions describe this dataset, and any use on another portfolio would need to be checked on that portfolio.

13.2 Finance definitions

Q8. What is the difference between your two default rates, and which one should we trust?

The default rate against all loans divides defaulted loans by every loan of the year. The default rate against resolved loans divides defaulted loans by the loans with a known outcome (paid off plus defaulted). Example, 2016: $1,122 / 43,368 = 2.6\%$ against all loans, but $1,122 / 6,037 = 18.6\%$ against resolved loans.

For comparing years, I trust the resolved-only rate more, because open loans cannot default yet and drag the first measure down. But it is not perfect either: for 2018 and 2019 only a few loans have resolved, and early resolvers are not representative. So I trust it for 2012-2017 and treat the last two years with caution.

Q9. Why do you measure the late rate against active loans and not against all loans?

Because only open loans can be late. A loan that is fully paid or already defaulted can no longer be in a grace period or 16-120 days late. If I divided by all loans, groups with many finished loans would look safe simply because their loans are finished. Overall, 5,614 of 176,075 active loans are late (3.19%); against all 270,299 loans it would be 2.08%, which would mix age with behaviour.

Q10. What is the loan-to-income ratio, and why did you pick 0.35 as a rule threshold?

It is the loan amount divided by annual income: a \$10,000 loan for someone earning \$50,000 gives 0.20. It shows how heavy the loan is relative to one year of earnings. In the data the highest fifth of loans starts at roughly 0.33 and their default rate is 28.5%, against 12.0% for the lowest fifth.

I chose 0.35 by eye from those buckets; it is not an optimised threshold. I say so in the limitations: it was tested on the same data it was chosen from, so it needs validation on new data before use.

Q11. What is your balance-to-income measure, and how is it different from credit utilisation?

Balance-to-income is total current balance across the borrower's accounts divided by annual income. A borrower with \$150,000 of balances and \$60,000 income scores 2.5. Credit utilisation is different: it is the balance on credit lines divided by the credit limits (for example \$3,000 owed on a \$10,000 card is 30%). We have no credit limits, and our balance figure includes mortgages, so it is a rough measure of leverage, not utilisation. I labelled it as a proxy everywhere.

Q12. Why did you treat Charged Off as default?

A charge-off is the lender's accounting decision that the loan will not be repaid and is written off as a loss. That is economically the same as default for a lender, and the data has only 12 loans labelled Default, so the two together (17,863 loans) form the defaulted group. I documented the mapping once in the view.

Q13. What is a grace period, and why is it counted as late?

It is a short window after a due date when a missed payment is not yet treated as seriously overdue. The loan is not current any more, so it is a warning sign, and I included it in the late group (1,638 loans). This is a judgement: without it, only 3,976 loans would count as late (802 at 16-30 days and 3,174 at 31-120 days), which is 2.26% of active loans instead of 3.19%. That is why I explain the definition openly.

13.3 Charts and findings**Q14. What is the main message of the Task 1 charts?**

Loan volume grew about twenty-fold between 2012 and 2019, and outcomes depend heavily on age: older loans are mostly finished (paid off or defaulted) and newer loans are mostly still running. Among 2012-2015 loans, 14.5-18.2% defaulted. The 2016 bar is odd, with 86% still active, which led to the careful treatment of default rates in Task 2.

Q15. The all-loans default rate falls sharply after 2015. Did lending quality improve?

No, or at least this data does not show it. The fall comes from loans not having finished yet. On the resolved-only basis, default moves from 20.2% (2015) to 18.6% (2016) and 19.8% (2017), essentially flat. The gap between the two lines (16.0 percentage points in 2016) is the share of unfinished loans. If someone read only the grey line, they would wrongly conclude that lending quality improved roughly sevenfold.

Q16. Does a higher interest rate cause more late payments?

The data cannot say that. What we observe is that loans with higher rates are late more often (0.6% in the 4-6% band to 14.9% in the 30-32% band). But the lender sets the rate according to the risk it sees, so the higher rate is largely a label for a riskier borrower. It is consistent with risk-based pricing working, not proof that the rate itself causes trouble. It is possible that a very high payment burden contributes too, but this analysis cannot separate the two.

Q17. Why are 36-month loans late more often than 60-month loans at the same rate?

In 11 of the 14 rate bands they are, and I did not investigate the reason, so I would not give a firm explanation. One possibility is that active 36-month loans are further along in their life than active 60-month loans, and lateness may depend on age; another is a difference in the type of borrower. I would need payment history and loan age to test it. It is best to say this plainly.

Q18. What do the five customer segments tell us, and why should we trust them?

They describe five recognisable types: small loans with low leverage; small loans with high existing balances; low income and stretched; 60-month higher-rate loans; and large loans with high income. The reason to give them some trust is that they were formed without looking at outcomes, yet their default rates differ by about threefold (10.6% to 32.3%), which suggests they capture real differences in risk. The reasons to be careful: the silhouette of about 0.20 shows they overlap heavily, and much of segment 4's high default is simply the known effect of a 60-month term.

13.4 Statistics and modelling**Q19. Why $k = 5$ clusters?**

Silhouette could not decide: it was about 0.20 for every k from 3 to 8 ($k = 2$ scores 0.269 but mostly separates 36-month from 60-month loans, which we already know). So I used other evidence. In a separate test with different random starts, $k = 5$ was very stable (similarity 0.993 between runs, against 0.975 for $k = 4$), and it produced a distinct group of customers with high balances relative to income that $k = 4$ does not have. It also gave five groups of manageable size (14% to 27% each). It is a judgement call and I present it as one.

Q20. A silhouette of 0.2 is low. Does that invalidate the clustering?

It does not invalidate it, but it changes how to describe it. A low silhouette means the customers form a continuous cloud rather than well-separated clumps, so the segments are a useful way of summarising the population, not natural boundaries. The fact that the segments' default rates still differ threefold shows the grouping carries real information. I say so in the notebook.

Q21. Why did you cap, take logarithms and standardise the features?

K-means measures distances, so it is sensitive to scale and to extreme values. Capping at the 1st and 99th percentile softens extremes without deleting customers (for example incomes above \$276,000 are set to \$276,000). The logarithm reduces the long tail of income and of the two ratios (skewness fell from 47.19 to 0.07 for income). Standardising, $z = (\text{value} - \text{mean}) / \text{standard deviation}$, puts every feature on the same scale, so that income measured in dollars does not overwhelm an interest rate measured in a few percentage points.

Q22. What does an AUC of 0.70 mean, and is it good?

AUC is the chance that, given one loan that defaulted and one that did not, the model gives the defaulted one the higher risk score: 0.5 is a coin flip and 1.0 is perfect. Our default model scores 0.697 and the late-payment model 0.674, on 30% of the loans that the model never saw. That is moderate: clearly better than chance, but far from a precise risk model. Grade alone scores 0.682, so nearly all the power comes from the lender's own grading.

Q23. Why did you leave grade out of the model?

The interest rate is derived from the grade, so the two carry almost the same information. Putting both in would make the model split the credit between them unpredictably and make the odds ratios hard to interpret. I kept the interest rate in the model and reported the grade's stand-alone strength (0.682) in the same table so nothing is hidden.

Q24. Why logistic regression and not a more powerful model?

Logistic regression is transparent: every characteristic gets an odds ratio that a business user can read, which suits a discussion about lending rules. The task also asked for insight, not maximum accuracy. A more flexible model such as gradient boosting might raise the 0.70 somewhat, and I would try it as a next step, but I would keep the simple model as the explainable benchmark.

Q25. What is an odds ratio?

The odds of an event are the chance that it happens divided by the chance that it does not. An odds ratio compares two situations. Our interest-rate odds ratio of 1.63 means that a loan whose rate is one standard deviation higher (about 4 to 5 percentage points) has odds of default 1.63 times as high, when the other characteristics are held fixed. Above 1 means riskier; below 1 means safer.

Q26. How did you avoid overfitting and data leakage?

The model was trained on 70% of loans and scored on 30% it had never seen, with the same share of events in both parts and a fixed random seed. The standardisation was learned from the training part only, so no information from the test loans leaked in. Clustering used no outcomes, so outcomes could be used afterwards as an independent check. The remaining weak point is the rule thresholds, which were tested on the same data they were chosen from; I say that in the limitations.

Q27. How reliable are your rule thresholds?

They are illustrative. I read the buckets and chose round numbers (grade E, 16%, 0.30, 0.35), then measured the effect on the same history. They show which kinds of rules work and what they cost, but before use they must be validated on later loans, and the money effect (interest lost against losses avoided) would have to be estimated.

13.5 Limitations and judgement

Q28. What are the biggest limitations of your analysis?

Five: (1) there is no payment history, so lateness is a snapshot; (2) statuses seem to come from different dates, so year-to-year comparisons need care; (3) only approved loans are present, so I cannot see how rejected applicants would behave; (4) default rates use resolved loans only; (5) rule thresholds were chosen and tested on the same data. Also, everything shown is an association, not a cause.

Q29. What would you do with more time or data?

Get payment history and credit limits to measure real delay and true utilisation; get rejected applications; find out the snapshot date of each cohort and use survival analysis (a method for loans that have not finished) for default timing; validate the rules on newer loans and estimate their money effect; and test a more flexible model against the simple one.

Q30. What could go wrong if the rules were used in real life?

The thresholds could be over-fitted to this history; the population could change over time so the rules drift; the flagged group still contains many good customers (about 60% of flagged loans do not default), so automatic rejection would lose good business; and any rule must be checked for fairness, so that it does not depend, directly or through proxies, on protected characteristics. I therefore recommend the rules as triggers for review and pricing, with monitoring.

Q31. Is there selection bias in the data?

Yes. Only loans that were approved and issued are in the data. We do not see applicants who were rejected, so we cannot know how they would have performed, and the relationships we found may differ in the full applicant population. For example, if the lender rejected most very risky applicants, the relationship between grade and default would look different from the situation in which all applicants are accepted.

Q32. What was the hardest challenge?

Two: the mismatch between the brief and the data, which forced workarounds for Tasks 3 and 4, and the snapshot inconsistency, which could have led to a wrong conclusion about lending quality. I dealt with the first by documenting each adaptation openly and with the second by presenting two default rates and labelling the unreliable years. A practical challenge was tooling: getting scikit-learn into the right Python environment in Hex, and a bug in which numbers moved by a tenth of a point between runs because rows arrived in a different order. I fixed it by sorting before bucketing and verified the fix with shuffled data.

Q33. What would you do differently?

I would ask the data provider for a data dictionary and the snapshot dates at the start; I would check earlier whether the outcome statuses were comparable across years; and I would include, from the beginning, a small cell in the notebook that reproduces the stability check I used for choosing k , so that this evidence is visible in the notebook and not only in a separate run.

Q34. How would you monitor the rules if they were introduced?

Track, month by month, the default rate and late rate inside and outside each rule; the share of applications flagged; the share of flagged customers who turn out fine; and the mix of grades, terms and loan-to-income in new business. A widening gap between flagged and unflagged rates means the rule still works; a narrowing gap or a shifting mix means it needs review.

Q35. Is there a fairness or regulatory concern with the rules?

Yes, and it is worth raising before being asked. I deliberately recommended rules on the lender's own grade, the loan term and loan-to-income, which relate to the loan and the borrower's ability to pay. I did not recommend rules on location, home ownership or other personal characteristics, which can act as proxies for protected characteristics. Any real rule would need review by compliance and legal teams and an assessment of its effect on different groups of customers.

13.6 Tools, reproducibility and security

Q36. Why BigQuery and Hex, and how did you connect them securely?

The brief asked for them, and they fit well: BigQuery stores and queries the data with SQL, and Hex combines SQL, Python, charts and written explanation in one shareable notebook. For security I did not use my personal login: I created a service account, a robot identity, with only read permissions (Data Viewer, Job User and Read Session User), so Hex can read but not change anything. Its key file is kept outside the project folder and never published.

Q37. How could someone reproduce your work?

All the code is in a public GitHub repository: the SQL (including the cleaned view and the validation queries), the Python cells, the scripts that profile the files and generate the table schemas, and documents describing every assumption. The raw data is not in the repository, but the README explains how to download it from Kaggle and rebuild the tables. Random seeds are fixed and the bucketing is sorted, so the numbers come out the same.

Q38. How did you protect credentials and personal data?

Secrets are never committed: the .env file, service-account keys, the raw data folder and the virtual environment are all listed in .gitignore, and I verified on GitHub that none of them are present. The customer identifiers in the data are already hashed, and I used loan_id as the readable handle. My commits use GitHub's no-reply email address, so my personal email address is not visible in the public history.

Q39. What is in the AGENT.md file in the repository?

It is a context file for AI assistants: it summarises the project, the decisions and definitions, the data facts, the file map, the environment quirks, my preferences for how to work, and the open items. If I open a new session, the assistant reads it and continues without needing everything re-explained.

Appendix A. Helper scripts used before and during loading

A.1 Profiling every CSV file

Before loading anything, this script read each CSV file and wrote a report (docs/schema_profile.md) with, for every column: its type, the share of blank values, the number of distinct values, the smallest and largest values, and the most common values. It is how we discovered the missing payments table, the stray header row, the text n/a, and the five spellings of application type.

scripts/profile_csvs.py

```
"""Profile every CSV in data/raw and write docs/schema_profile.md.

Read-only on the data. For each file: row count, duplicate rows, and per-column
dtype, null %, distinct count, min/max (numeric), and top values (low-cardinality).
"""
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
RAW = ROOT / "data" / "raw"
OUT = ROOT / "docs" / "schema_profile.md"

LOW_CARD = 15 # show top values when a column has at most this many distinct values

def profile_column(s: pd.Series) -> dict:
    n = len(s)
    nunique = s.nunique(dropna=True)
    row = {
        "column": s.name,
        "dtype": str(s.dtype),
        "null_%": round(100 * s.isna().mean(), 2),
        "distinct": nunique,
        "min": "",
        "max": "",
        "sample / top values": "",
    }
    if pd.api.types.is_numeric_dtype(s):
        row["min"], row["max"] = s.min(), s.max()
    elif s.dtype == object or str(s.dtype).startswith("str"):
        non_null = s.dropna().astype(str)
        if len(non_null):
            row["min"], row["max"] = non_null.min(), non_null.max()
    if nunique <= LOW_CARD and nunique > 0:
        vc = s.value_counts(dropna=False).head(LOW_CARD)
        row["sample / top values"] = "; ".join(f"{k}: {v}" for k, v in vc.items())
    else:
        row["sample / top values"] = "; ".join(map(str, s.dropna().unique()[:3]))
    return row

def main() -> None:
    files = sorted(RAW.rglob("*.csv"))
    lines = ["# Schema profile (auto-generated)\n"]
    if not files:
        raise SystemExit(f"No CSV files found under {RAW}")
    for f in files:
        df = pd.read_csv(f, low_memory=False)
        lines.append(f"\n## `{f.relative_to(RAW)}`\n")
        lines.append(f"- rows: **{len(df):,}** | columns: **{df.shape[1]}**")
        lines.append(f"- fully duplicated rows: **{df.duplicated().sum():,}**\n")
        prof = pd.DataFrame([profile_column(df[c]) for c in df.columns])
        lines.append(prof.to_markdown(index=False))
        print(f"profiled {f.name}: {len(df):,} rows, {df.shape[1]} cols")
    OUT.write_text("\n".join(lines), encoding="utf-8")
    print(f"wrote {OUT}")

if __name__ == "__main__":
    main()
```

- **pd.read_csv(f, low_memory=False)** reads a file into a table; low_memory=False stops pandas from guessing types chunk by chunk.
- **profile_column** computes, for one column: dtype (its type), null_% (100 x average of 'is missing'), distinct (count of different values), min and max, and the most frequent values when there are 15 or fewer distinct values.

- `df.duplicated().sum()` counts rows that are complete copies of another row (none were found).
- The result is written as a Markdown table so it can be read in any editor and stored in the repository.

A.2 Generating the BigQuery table schemas

This script holds the intended type of every column of the six tables and checks that the columns are in the same order as the real file headers (BigQuery matches columns by position). It then writes one small JSON file per table (in `sql/schemas`), ready to be pasted into the BigQuery console.

scripts/make_schemas.py

```
"""Generate BigQuery schema JSON files (sql/schemas/<table>.json) for the 6 raw CSVs.

Paste each file's contents into the BigQuery console: Create table -> Schema -> "Edit as text".
Columns are loaded by POSITION, so the order below must match each CSV header exactly;
this script asserts that before writing anything.
"""

import json
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parent.parent
RAW = ROOT / "data" / "raw"
OUT = ROOT / "sql" / "schemas"

# Raw layer: load faithfully, clean later in views. Dates like 'Jun-13' cannot be
# loaded as DATE by BigQuery, so they stay STRING here and are parsed in the staging view.
SCHEMAS = {
    "customer": {
        "customer_id": "STRING", "emp_title": "STRING", "emp_length": "STRING",
        "home_ownership": "STRING", "annual_inc": "FLOAT", "annual_inc_joint": "FLOAT",
        "verification_status": "STRING", "zip_code": "STRING", "addr_state": "STRING",
        "avg_cur_bal": "FLOAT", "Tot_cur_bal": "FLOAT",
    },
    "loan": {
        "loan_id": "INTEGER", "customer_id": "STRING", "loan_status": "STRING",
        "loan_amount": "INTEGER", "state": "STRING", "funded_amount": "INTEGER",
        "term": "STRING", "int_rate": "FLOAT", "installment": "FLOAT",
        "grade": "STRING", "issue_d": "STRING", "issue_date": "STRING",
        "issue_year": "INTEGER", "pymnt_plan": "BOOLEAN", "type": "STRING",
        "purpose": "STRING", "description": "STRING", "notes": "STRING",
    },
    "loan_with_region": {"loan_id": "INTEGER", "loan_amount": "INTEGER", "region": "STRING"},
    "loan_count_by_year": {"issue_year": "INTEGER", "loan_count": "INTEGER"},
    "loan_purposes": {"purpose": "STRING"},
    "state_region": {"state": "STRING", "subregion": "STRING", "region": "STRING"},
}

def main() -> None:
    OUT.mkdir(parents=True, exist_ok=True)
    for table, cols in SCHEMAS.items():
        header = list(pd.read_csv(RAW / f"{table}.csv", nrows=0).columns)
        assert header == list(cols), f"{table}: CSV header {header} != schema {list(cols)}"
        fields = [{"name": n, "type": t, "mode": "NULLABLE"} for n, t in cols.items()]
        (OUT / f"{table}.json").write_text(json.dumps(fields, indent=2) + "\n", encoding="utf-8")
        print(f"ok {table}: {len(fields)} columns")

if __name__ == "__main__":
    main()
```

- **SCHEMAS** is a dictionary: table name, then column name and type. The types use the older names accepted by the console (INTEGER, FLOAT, BOOLEAN, STRING).
- **assert header == list(cols)** stops the script with an error if the file's columns and the schema's columns differ in name or order.
- **json.dumps(fields, indent=2)** writes the list of {name, type, mode} entries; mode NULLABLE means that a value may be missing.

Appendix B. The GitHub repository

The whole project is published at <https://github.com/lukatcheishvili/fintech-loan-analysis> (public). The repository contains 32 files. Files that must never be public are listed in `.gitignore` and were verified to be absent from GitHub: the `.env` file (which holds a token), the raw data folder, the virtual environment, key files, and the assignment PDF.

Path	What it holds
README.md	Public overview: headline findings, data limitations, layout and how to reproduce.
AGENT.md and CLAUDE.md	Project context for AI assistants (CLAUDE.md simply imports AGENT.md so it loads automatically in Claude Code).
sql/00_staging.sql	The cleaned view <code>v_loans</code> (every definition).
sql/01_validate_load.sql, sql/02_validate_staging.sql	Validation queries with the expected values.
sql/10_*, 11_*, 20_*, 30_*, 40_*	Queries for Tasks 1 to 4.
sql/schemas/*.json	The six BigQuery table schemas.
notebooks/task3_*.py to task5_*.py	The Python cells used in Hex, kept as scripts.
scripts/profile_csvs.py, scripts/make_schemas.py	Helper scripts (Appendix A).
docs/assumptions.md	Every judgement call and cleaning rule.
docs/talk_track.md	Preparation notes for the call.
docs/schema_profile.md	Auto-generated profile of the source files.
Pave Bank - Fintech Loan Analysis.ipynb	Export of the Hex notebook (SQL cells appear as commented code; native Hex charts are not included in the export).

Reminder: items still open in the notebook

Text clean-ups recorded in AGENT.md (some notes contain outdated numbers or internal wording), the lending-rules cell to add to the Hex notebook, and re-exporting the notebook file afterwards. The numbers in this guide are the verified ones.

Appendix C. Key numbers at a glance

C.1 The dataset

Fact	Value
Loans (and customers)	270,299, one loan per customer; issued 2012 to December 2019
Total principal	\$4,166,072,400 (about \$4.17 billion); average loan \$15,413; median \$13,200
Interest rate	5.31% to 30.99%; mean 13.08%; median 12.61%
Terms	36 months: 189,772 loans; 60 months: 80,527 loans
Outcome groups	paid_off 76,361; defaulted 17,863; active 176,075
Resolved / late loans	resolved 94,224; late 5,614 (3.19% of active loans)
Overall default rate (resolved loans)	18.96%

C.2 Results by task

Task	Headline numbers
1	Volume 2,594 (2012) to 51,737 (2019). Default share of loans 14.5-18.2% for 2012-2015 and 2.6% for 2016. 86.1% of 2016 loans still active.
2	All-loans default rate 18.19% (2015) to 2.59% (2016); resolved-only 20.22% to 18.59%. Average rate 12.60-14.54%; average principal about \$13.7k-\$16.5k.
3	Late rate 0.6% (4-6% rate band) to 14.9% (30-32%), and the pattern holds within each term. 36-month loans are late more often than 60-month loans in 11 of 14 bands.
4	k = 5 segments; default 10.6% to 32.3%; late 2.5% to 4.0%; silhouette about 0.20 for k = 3 to 8; stability 0.993 at k = 5.
5	Default by grade 5.8% (A) to 48.1% (G); model AUC 0.697 (default) and 0.674 (late); rules: grade E-G flags 10.2% of resolved loans at 40.7% default versus 16.5%.